

Chapter 1

Introduction

This chapter gives a general idea of the research problem, why it is important, and how deep learning can help with problems in finding coconut diseases. It talks about the project's goals, reasons for doing it, and expected results. It also gives a brief overview of the methodology used and the report's overall structure.

1.1 Introduction

The cultivation of coconuts is an important part of farming in many tropical areas, but its productivity is getting worse because of a number of serious diseases, such as Bud Rot, Leaf Rot, Yellow Leaf Disease, and Stem Bleeding. These diseases can make plants very sick, lower fruit production, and cost farmers money in the long run. Studies show that coconut diseases are still spreading quickly because of changes in the weather, bad disease management, and late diagnosis. This makes it very hard to find them early in many countries where coconuts are grown [1], [3], [7].

Traditional diagnosis depends on manual observation, which can be biased, inconsistent, and reliant on the availability of experts. The visual similarities between disease symptoms and the effects of environmental factors make it even harder to find the right ones [4], [8]. Because of this, farmers often don't notice diseases until later, which lets infections spread across plantations.

Deep learning is a strong alternative for automated disease detection through image-based classification. CNN architectures like ResNet, DenseNet, MobileNet, and EfficientNet have shown that they can recognize agricultural diseases very well because they can extract features very well [10], [14], [16]. Recent studies have underscored the significance of lightweight models for effective field deployment and the necessity of explainable AI to enhance trust and interpretability in agricultural systems [20, 22].

This study suggests a deep learning solution for identifying five significant coconut diseases through the utilization of four pretrained models: DenseNet121, ResNet50, MobileNetV2, and MobileNetV3. These models are trained and tested on a dataset of 5,000 labeled images. Grad-CAM is used as an explainable AI method to show disease-specific areas in leaf images. The goal is to create a disease detection system that is accurate, can be used by many farmers, and helps with early intervention and long-term coconut production.

1.2 Motivation

Coconut farming supports millions of people, but disease outbreaks keep causing big losses for the economy. Farmers in rural areas with few resources have trouble getting timely disease diagnoses because they can't easily get to agricultural experts and it's hard to spot early signs. A delay in finding a disease can quickly spread it across plantations, which can hurt productivity in the long run. Deep learning is a promising alternative because it lets you quickly, consistently, and accurately recognize diseases using pictures of leaves. Models like ResNet50 and DenseNet121 can find very small visual patterns that people often miss when they look at things. On the other hand, lightweight architectures like MobileNetV2 and MobileNetV3 make it possible to make diagnostic tools that work on mobile devices. Adding explainable AI makes the system even more open, so that everyone involved can see why a prediction was made. The goal of this project is to make an inexpensive and scalable coconut disease detection system that meets the needs of tropical farming communities. It also gives the researcher a chance to learn more about deep learning, computer vision, and how to use AI in real life.

1.3 Objectives

Based on our research, our main objectives are mentioned as follows:

- To create a system based on deep learning that can find and categorize the most common diseases of coconut trees.
- To use transfer learning on pretrained models (DenseNet121, ResNet50, MobileNetV2, MobileNetV3) to make classification faster and more accurate.
- To check the models' accuracy, precision, recall, F1-score, and confusion matrices.
- To use explainable AI (Grad-CAM) to show disease-specific features.
- To use Grad-CAM to see how easy it is to understand each model and see how well the highlighted areas match real disease symptoms.
- To compare how well different models work and find the best way to build a model for detecting coconut disease.
- To look into how scalable and useful lightweight models are for use in real time or on mobile devices.
- To offer a useful and affordable solution that helps with early diagnosis and lowers crop losses.
- To find out how well each deep learning architecture works for real-time or mobile-based use.

- To set up a way to compare performance that finds the best architecture for field-level coconut disease detection that is both practical and scalable.

1.4 Research Questions

Here are three research questions derived from based on our research:

RQ1: How well can deep learning models use image-based data to tell the difference between coconut leaf diseases in real-world farming situations?

RQ2: What are the differences in performance, computational cost, and interpretability between the chosen models, and which architecture is best for use in the field?

RQ3: How well can explainable AI methods like Grad-CAM show disease-specific leaf areas and make coconut disease classification models easier to understand?

1.5 Project Outcome

The expected outcomes of this project include:

- An operational deep learning framework proficient in categorizing five principal coconut diseases.
- Performance benchmarks for DenseNet121, ResNet50, MobileNetV2, and MobileNetV3 using a number of different evaluation metrics.
- Explainable AI visualizations (Grad-CAM) that show important parts of an image that affect model predictions.
- A comparison that shows the pros and cons of both heavy and lightweight architectures.
- A possible basis for creating a disease detection tool that works in real time and is easy to use on mobile devices for farmers.
- A step toward smart agriculture by making it easier to find diseases, manage crops in a way that is good for the environment, and lower economic losses.

1.6 Organization of the Report

Chapter 1: The Beginning

In this chapter, the author talks about the research's background, goals, scope, and significance. It also talks about why the study was done and how deep learning models and explainable AI (X-AI) fit into the disease-detecting model of agriculture.

Chapter 2: Background

This chapter is a review of the literature on the research topic. It talks a lot about how to find diseases in agriculture, how to use deep learning methods, and how the models fit into the right procedures.

Chapter 3: How the Research Was Done

This chapter explains the research design, the methods used, and how data was collected. It also talks about the model architectures, training procedures, hyperparameters, and the Grad-CAM technique, as well as how these steps have been used to make classifications more accurate.

Chapter 4: Implementation and Results

This chapter explains how to set up and configure the system, including the hardware, software, and software tools used to train and test the models. The results of the model performance are shown, and the results of the models are compared to each other in terms of accuracy, loss, precision, recall, and f1-score.

Chapter 5: The difficulty of standards and design in engineering.

This chapter will talk about the project's engineering standards, such as those for hardware, software, and communication. It also talks about the design problems that came up when making the models useful for real-time disease detection and the need to find a balance between the models' accuracy and the hardware resources they would need to work and be used in places with few resources.

Chapter 6: Final Thoughts

The final chapter summarizes the study's findings, critiques the conducted work, and delineates potential avenues for further research to augment and expand the study. It also points out the system's weaknesses and suggests ways to fix some of the problems in future versions of the disease detection system.

Chapter 2

Background

This chapter gives the information needed to understand the ideas, methods, and previous research on using deep learning to find coconut disease. It talks about basic ideas, looks at important literature, gives an overview of similar projects, points out gaps in research, and shows how this project fits into the work that has already been done in the field.

2.1 Introduction

To keep crops healthy and make sure agriculture can last, it's important to find plant diseases early. Coconut trees, which are grown a lot in tropical countries, are very susceptible to diseases like Bud Rot, Leaf Rot, Yellow Leaf Disease, and Stem Bleeding. These diseases hurt plantations a lot, which lowers the amount of crops they can grow and hurts farmers' livelihoods. In large-scale farming settings, traditional diagnosis depends on manual inspection, which takes a lot of time, isn't always reliable, and is often wrong. Deep learning has changed how diseases are found in pictures by making it possible to automatically classify them with high accuracy and scalability. Transfer learning models like DenseNet, ResNet, and MobileNet, as well as Convolutional Neural Networks (CNNs), have shown that they can do a great job of recognizing agricultural diseases. These models can find small changes in the texture and color of infected leaves, which makes them good for finding diseases early. This chapter talks about how deep learning has grown in plant pathology, looks at current studies, and lays the groundwork for the suggested coconut disease detection system.

2.2 Literature Review

Deep learning has become the primary technique used to automatically find crop diseases because it can pull out important features from complicated agricultural images. Multiple studies have utilized convolutional neural networks (CNNs) and sophisticated deep learning architectures to identify diseases in coconut leaves, yielding encouraging outcomes.

Subbaian et al. [1] utilized YOLOv4 for the detection of coconut leaf diseases, attaining notable results with an F1-score of 88% and a mean Average Precision (mAP) of 85%. This illustrates that contemporary object detection architectures can effectively manage variations in lighting and background conditions. Singh et al. [2] created a deep learning framework that combined custom CNN models with pretrained architectures like MobileNet and InceptionResNetV2. This framework was able to find stem bleeding, leaf blight, and Red Palm Weevil infestations with

96.94% accuracy. Brar et al. [3] employed the ResNeXt50 architecture to classify Coconut Leaf Spot disease into six severity levels, attaining an accuracy of 91.77%, thereby demonstrating the effectiveness of deeper CNN architectures for detailed disease evaluation.

Datasets that support the model also have a big impact on how well it works. Thite et al. [4] presented a standardized dataset comprising 5,798 images of coconut leaf diseases across five principal categories, thereby promoting reproducible research and facilitating comparative model assessments. Review papers, like Nivethitha et al. [5], talk about how more and more people are using CNN-based frameworks to find plant and coconut diseases. They also point out problems with current datasets and the need for consistency across different environmental conditions. Other studies suggest unified frameworks that amalgamate various image processing and deep learning methodologies. Maray et al. [6] proposed the AIE-CTDDC model, employing Bayesian fuzzy clustering for segmentation and integrating CapsNet with GRU for classification. Their hybrid method worked better than others, showing that combining traditional segmentation with deep learning can work well. Muthulakshmi et al. [7] tested ResNet-50, EfficientNet-B0, and DenseNet-201 with different activation functions and found that the choice of activation function can have a big impact on how well a model works.

Early detection is still possible with traditional image processing. George et al. [8] attained 80.36% pixel accuracy in identifying stem bleeding disease through color thresholding, surpassing traditional methods. But these methods don't work well when the lighting or background changes, which shows how much better deep learning is. Deep learning has also been used to find pests, like Kavithamani and UmaMaheswari's [9] work on finding whiteflies on coconut leaves. This shows how AI is becoming more important in pest control. Several ML-based methodologies have been investigated. Karegowda et al. [10] utilized GLCM and SIFT features in conjunction with conventional classifiers, attaining an accuracy of up to 100% with SIFT features, albeit necessitating meticulous feature engineering. Wijethunga et al. [11] developed a comprehensive framework that amalgamates disease detection, environmental monitoring, and growth prediction, illustrating the adaptability of machine learning in agricultural decision support systems. Santhi and Murugan's Cocoscan [12] is an example of a system that combines deep learning with XAI using InceptionV3. This shows how automated diagnosis of coconut diseases can be made more understandable.

Many deep learning architectures have shown to be very good at finding coconut leaf disease. In Anitha and Santhi's study [13], DenseNet-121 got about 99% right, and Toradmalle et al. [14] showed that InceptionResNetV2 and DenseNet121 were also good at finding diseases early. Studies on general plant disease detection using DCNNs have shown accuracies of up to 99.97% [15], which sets standards for classifying agricultural diseases. Research using CNNs, like the ones by Kumar et al. [16] and Lu et al. [17], shows that CNNs are good for detecting diseases in real time. The goal of hybrid architectures is to make things as accurate and efficient as

possible. Bedi and Gole [18] put forward a CAE-CNN hybrid model that kept high performance while lowering the cost of computing. Falaschetti et al. [19] utilized a lightweight CNN on embedded hardware for mobile agricultural applications. Models based on vision transformers are becoming strong competitors to CNNs. Thakur et al. [20] integrated CNNs and ViTs for the classification of plant diseases, demonstrating enhanced accuracy and interpretability.

More advanced uses of agriculture include using Mask R-CNN and YOLO-based models to find Weligama Coconut Leaf Wilt early on, which works well in real-world settings [21]. Ensemble learning and stacking techniques have been utilized, as demonstrated by Sharma et al. [22], attaining 98.4% accuracy in the detection of coconut leaf disease. Wijesinghe et al. [23] looked into UAV-based detection systems that use multispectral imaging and SVMs. This showed that coconut plantations can be remotely sensed in other ways. ResNet50 and other models like it continue to do well at finding multiple types of coconut leaf diseases. Kaur et al. [24] found that it was 98.7% accurate. Gopalakrishna and Lingaraju [25] also showed that models based on MobileNetV2 can accurately predict coconut disease. In addition to studies on coconut plantations, many papers on using CNNs and Faster R-CNN to find rice leaf disease [26, 27] give us basic information and ideas for making strong detection frameworks. Research utilizing transfer learning and ensemble fusion exhibits elevated accuracy and generalization across various crops as well [28, 29, 30, 31].

These results show that deep learning has made a lot of progress in finding coconut leaf disease and plant pathology. But there is still a need for better generalization, more diverse datasets, lightweight models that can be used in real time, and explainability to make it easier to use in real life.

Table 2.1: Summary of Literature Reviewed.

Author (s)	Year	Title	Methodology	Key Findings
Subbaian et al. [1]	2024	Coconut leaf disease detection	YOLOv4	Achieved 88% F1-score and 85% mAP for leaf disease detection.
Singh et al. [2]	2021	Coconut disease and pest detection	CNN, MobileNet, InceptionResNetV2	Custom CNN achieved 96.94% accuracy.
Nivethitha et al. [5]	2022	Review on coconut disease detection	Deep Learning Review	Summarized CNN-based methods and challenges.
Muthulakshmi et al. [7]	2025	Coconut disease classification	DenseNet201, ResNet50, EfficientNetB0	Achieved up to 99.85% accuracy.

George et al. [8]	2023	Early detection of stem bleeding	Color thresholding	Achieved 80.36% pixel accuracy.
Thite et al. [4]	2023	Coconut tree disease dataset	Dataset creation	5,798 labeled images across 5 disease categories.
Wijethunga et al. [11]	2023	Coconut plant disease identification and management	Machine Learning & Image Processing	Developed an integrated system combining environmental monitoring & ML-based disease classification.
Santhi & Murugan [12]	2025	Cocoscan: Precision diagnostics for coconut leaf disease	InceptionV3, Web App, XAI	Real-time disease diagnosis with XAI-based visual explanations for user trust
Pandian et al. [15]	2022	Plant disease detection using deep CNN	14-layer custom DCNN	Achieved 99.97% accuracy with large augmented dataset across 58 disease classes.
Lu et al. [17]	2021	Review of CNN for plant disease classification	Survey/Review	Summarized state-of-the-art CNN architectures and highlighted research gaps.
Falaschetti et al. [19]	2022	Vision transformer + CNN hybrid for plant disease classification	ViT + CNN Hybrid	Improved accuracy and transparency using a lightweight ViT-CNN architecture.
Vidhanaarachchi et al. [21]	2025	Early diagnosis of coconut leaf wilt & caterpillar infestation	CNN, Mask R-CNN, YOLO	Achieved 90–97% disease accuracy and >95% caterpillar detection rate.

2.2.1 Similar Applications

Deep learning-based disease detection has been widely applied in agriculture, yielding insights relevant to coconut plantations. Mohamed et al. made a mobile app with Faster R-CNN that could find pests in real time with 98% accuracy. This proves that you can use mobile apps in the field. Li et al. used CNN architectures to find pests in rice and got 82.4% accuracy. This shows how important it is to have good ways to extract features in farming. Liu et al. also used SSD with MobileNet to find pests in real time, and they were right 90.6% of the time. This shows that lightweight models can work well when there aren't many resources available. Nanni et al. [18] achieved 92.4% accuracy utilizing multiple CNN models, whereas Li et al. employed YOLOv5-S for real-time pest detection with 89.6% accuracy, illustrating the significance of balancing speed and performance. These apps show how deep learning is becoming more important for making tools for farming that work well and can be used by a lot of people. A lot of these tools have a direct impact on the decisions made about how to find coconut diseases.

2.3 Gap Analysis

Finding coconut leaf disease is now far better, but there are still some things that need to be fixed. A lot of recent studies use limited datasets that don't have enough variety in terms of lighting, age, disease severity, and environmental context, which makes it harder for models to work in other situations. Most CNN-based models do well in controlled tests, but they don't do as well in the real world. There aren't many lightweight models that can be used on mobile devices or at the edge, even though on-site diagnosis is important for plantations. This is another big gap. Some studies look at advanced architectures like CapsNet or transformers, but not many look at explainable AI methods that could make people trust AI more, which is important for getting people to use AI in farming. Ensemble learning and hybrid methods are getting better, but there aren't many tests that look at different types of coconut diseases yet. Moreover, there is insufficient research on integrating detection, classification, and severity estimation into a unified model. The suggested system fills these gaps by training advanced deep learning models on a structured five-class dataset, using explainable AI (Grad-CAM), and putting a lot of emphasis on real-time performance and model reliability.

Table 2.2: Gap Analysis

Features	Existing Coconut Systems	Proposed System
Multi-disease classification	No	Yes
Balanced dataset	No	Yes
Heavy + lightweight model comparison	No	Yes
Explainable AI (Grad-CAM)	No	Yes

Mobile/real-time feasibility	No	Yes
Early-stage detection	No	Yes
Comparison across multiple pretrained architectures	No	Yes
Scalability for large-scale deployment	No	Yes

2.4 Summary

This chapter went over some important background information about how to find coconut diseases and how deep learning can help automate agricultural diagnostics. Many studies show that CNNs, transfer learning, hybrid frameworks, and even transformers work well for classifying plant diseases. Previous research demonstrates robust performance across diverse architectures; however, it also uncovers enduring challenges, including inadequate dataset diversity, performance variability under real-world conditions, and the absence of integrated, transparent, and lightweight solutions for field deployment. The review of similar applications and gap analysis shows that the field is moving quickly, but we still need models that are accurate, easy to understand, and efficient to use. The insights collected here are the basis for the methodology and system that will be proposed in the next chapters.

Chapter 3

Research Methodology

This chapter describes the research methods used to create, test, and improve the coconut disease detection system. It describes the methodological framework used for preparing the dataset, choosing the model, preprocessing techniques, training procedures, Explainable AI integration, and evaluation strategies. It also explains why certain architectures, like DenseNet121, EfficientNet-B4, and ensemble fusion models, were chosen to make disease classification more accurate and faster.

3.1 Methodology

This study employed a systematic research methodology to develop a dependable and scalable classification system for coconut leaf diseases. First, a lot of research on using deep learning to diagnose plant diseases was looked at. This included CNN architectures, lightweight models for mobile deployment, ensemble learning, and Explainable AI methods like Grad-CAM. The review helped us find some of the main problems with previous work, like datasets that aren't balanced, diseases that look similar but are misclassified, and models that aren't easy to understand. This methodology aims to enhance accuracy via fusion models, decrease computational demands through lightweight networks, balance the dataset, and incorporate explainability to bolster confidence in predictions. The methodology also includes testing performance, comparing different models, and getting ready for real-time use.

3.1.1 Overview

The study presents a deep learning-based system engineered to categorize various coconut leaf diseases through an enhanced framework of preprocessing, augmentation, transfer learning, and model evaluation. The first step in the process is to gather and prepare a dataset of pictures of healthy and sick coconut leaves. To make learning less biased, the dataset is cleaned and balanced. Preprocessing involves reducing noise, changing the size of the samples, normalizing them, and adding more samples to make them more diverse and avoid overfitting. During the experiments, we looked at a number of different model architectures. Some were heavy, like DenseNet121, which helped us get better feature extraction, and others were light, like EfficientNet-B4, which helped us deploy on mobile devices or in places with few resources. Using Grad-CAM to show disease-specific areas and check the model's reliability, explainable AI is built in. The whole pipeline makes sure that the system can be used in real time, in places with limited resources, and for making practical decisions in farming.

3.1.2 Proposed Methodology

The proposed methodology adopts a multi-stage deep learning framework for accurate and computationally efficient coconut disease classification. The process begins with dataset preprocessing, where images are normalized, resized, cleaned, and augmented with rotations, brightness shifts, flips, scaling, shearing and zoom variations. This enhances robustness and mitigates dataset imbalance. The training stage involves experimenting with multiple architectures including DenseNet121, MobileNetV2, MobileNetV3, ResNet50, and Xception. Heavy models are used to extract complex visual features, while lightweight models ensure fast inference. Training is performed using transfer learning, fine-tuning, and regularization to accelerate convergence. Evaluation metrics include accuracy, precision, recall, F1-score, and confusion matrices. Explainable AI using Grad-CAM is applied after training to highlight the specific leaf regions influencing each prediction, confirming that the system makes medically meaningful decisions. The complete workflow of the proposed methodology is illustrated in Figure 3.1.

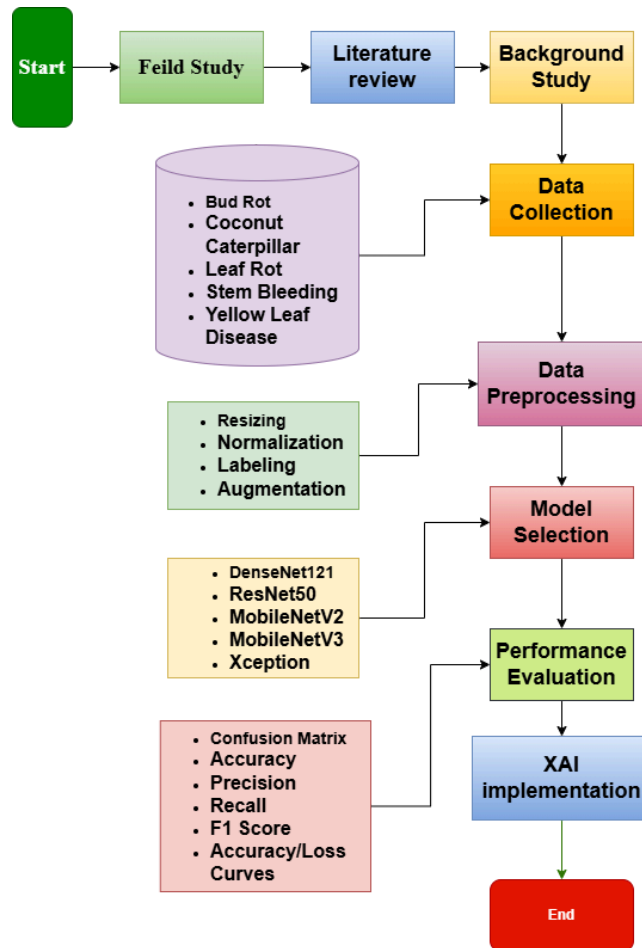


Figure 3.1: Methodology diagram

3.2 Detailed Methodology and Design

This part describes the whole process that was used to make the coconut tree disease detection system. It talks about how the dataset was gathered, cleaned, preprocessed, augmented, and split up for model training. It also gives a final summary of the prepared dataset that was used in the experiments. While developing the model, a number of other options were looked at. We first looked at traditional machine learning classifiers like SVM and Random Forest, but we didn't like them because they couldn't capture the complex visual features of diseased leaves very well. We also looked at simple CNN architectures, but they didn't work well when there was a lot of similarity within the same class. The final choice was deep learning models like DenseNet121, ResNet50, MobileNetV2, and MobileNetV3 because they are very good at learning features. We made adjustments to the models to make them more accurate. Grad-CAM was added to make the system easier to understand, which made it more reliable and better for use in real-world farming. This design makes sure that accuracy, interpretability, and computational efficiency are all in balance.

3.2.1 Data Collection

This research's dataset was gathered from several publicly accessible online repositories and open-source plant disease image databases. Because the raw data had mixed classes, different resolutions, and some samples that weren't useful, all of the images were looked over by hand to make sure they were good for classifying coconut diseases. After the cleaning phase, this study settled on five main types of coconut disease: Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease. These classes show the most common and harmful coconut leaf problems that happen on farms.



Figure 3.2: 5 images from each class.

3.2.2 Data Preprocessing and Cleaning

3.2.2.1 Data Cleaning

The original dataset had duplicate images, samples that were very low quality, and images that weren't relevant. A combination of manual inspection and automated validation checks got rid of these. Only pictures that clearly showed one of the five disease categories were kept. This made sure that the data was very accurate and lowered the chance of misclassifying during training.

3.2.2.2 Data Resizing

All of the images were resized to 224×224 pixels so that the dataset would be the same size and meet the input needs of pretrained CNN models. This resolution strikes a good balance between being efficient with computers and keeping important disease-related features like spots, textures, lesions, and color changes.

3.2.2.3 Normalization

To make the numbers more stable and speed up model convergence, pixel values were normalized to a fixed range (usually 0–1 or the mean and standard deviation of ImageNet). Normalization makes sure that the contrast and lighting are always the same, which helps models learn patterns better.

3.2.2.4 Dataset Balancing

To avoid bias in the model toward any dominant class, the dataset was balanced by making sure that all five disease categories were equally represented. Only where needed were oversampling and augmentation used. This step helps the model treat each class equally important when it is being trained.

3.2.2.5 Data Augmentation

A wide range of augmentation methods were used to make the dataset more diverse and the model more robust. These were things like translation, rotation, shearing, scaling, flipping horizontally, and changing the contrast. The goal was to make changes to the environment that would happen in real life, like changing the angle of the leaves, the lighting, and the camera angle. This step made the dataset work better by adding about 1000 images to each class.

3.2.2.6 Training and Testing Split

The dataset was split into 80% for training and 20% for testing after preprocessing and adding more data. The training set was used to fine-tune the model's parameters, and the testing set was only used to see how well it worked in the end. This split makes sure that the assessment is accurate and that no data leaks.

3.2.2.8 Final Dataset Overview

The final curated dataset has five balanced classes: Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease. There are 1000 augmented and preprocessed images in each class. All of the images have the same size of 224×224 pixels and the same pixel values. The dataset is clean, varied, and well-organized, which makes it a good choice for training many different deep learning models and checking their performance over time.

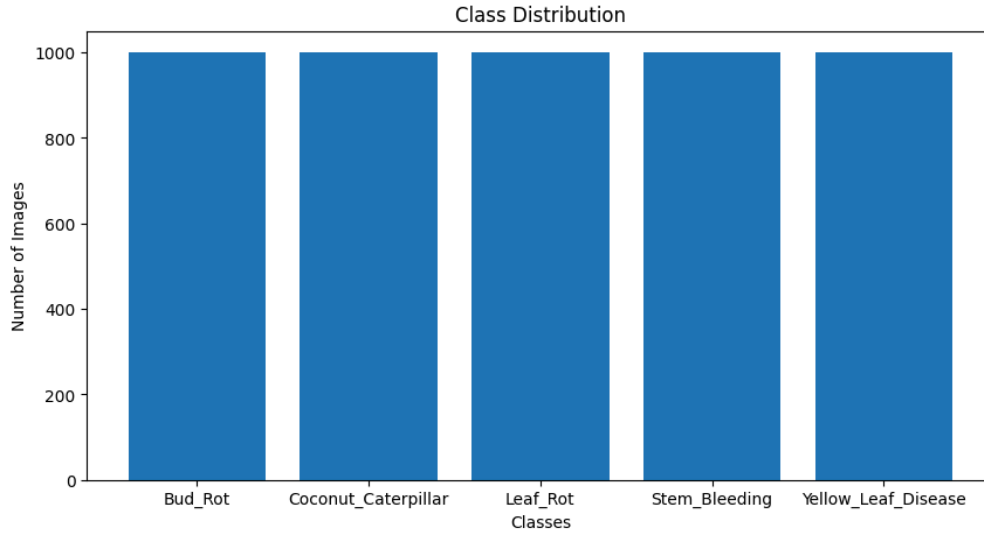


Figure 3.3: Class distribution of the dataset.

3.2.3 Model Architecture

3.2.3.1 Convolutional Neural Network (CNN)

The Convolutional Neural Network (CNN) is one of the most important deep learning architectures for classifying images. This is because CNNs can learn spatial features from visual data on their own. Convolutional layers in CNNs look for patterns like edges, textures, and shapes. Pooling layers make the spatial dimensions smaller, and fully connected layers sort the features that were found. The basic structure usually has a Conv-Pool sequence that goes through itself a few times, and then there are dense layers at the end. By stacking convolution and pooling operations, CNNs learn hierarchical representations. They begin with simple, low-level features and progress to more intricate, high-level structures. Because of this, CNNs are great for tasks that use pictures of plant leaves, where disease symptoms might show up as small changes in color, strange textures, or small lesions. In this study, CNNs are the best choice for the first steps in finding coconut disease because they can learn features well and run quickly.

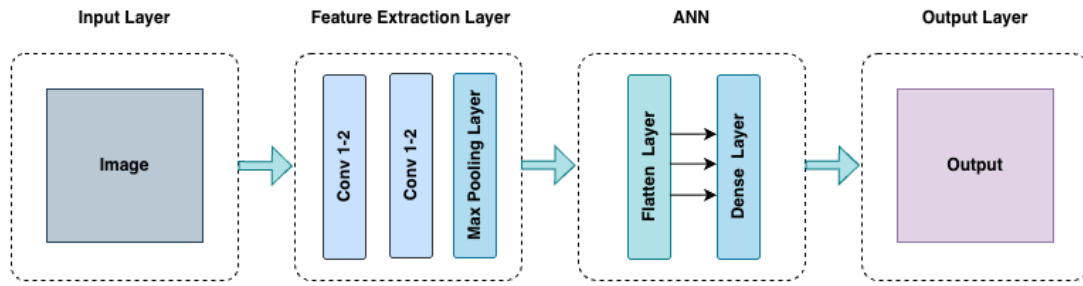


Figure 3.4: CNN Architecture

3.2.3.2 Transfer Learning

This study employs transfer learning to address the constraints of limited size and variability in coconut leaf disease datasets. To avoid overfitting, pretrained models like DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception are used as feature extractors instead of training deep neural networks from scratch. The large-scale ImageNet dataset was used to train these models, which means they already know a lot about colors, shapes, textures, and edges. The models quickly learn to recognize coconut-specific disease patterns by keeping their convolutional bases and changing the last classification layers. This means they need fewer training samples and less time to compute.

This method guarantees quicker convergence, higher accuracy, and superior generalization across diverse lighting conditions and leaf appearances. It also lets the study compare lightweight architectures with heavier ones, which helps the goal of finding efficient and useful solutions that can be used in the field. Overall, transfer learning is very important for making coconut disease detection models that work well in the real world.

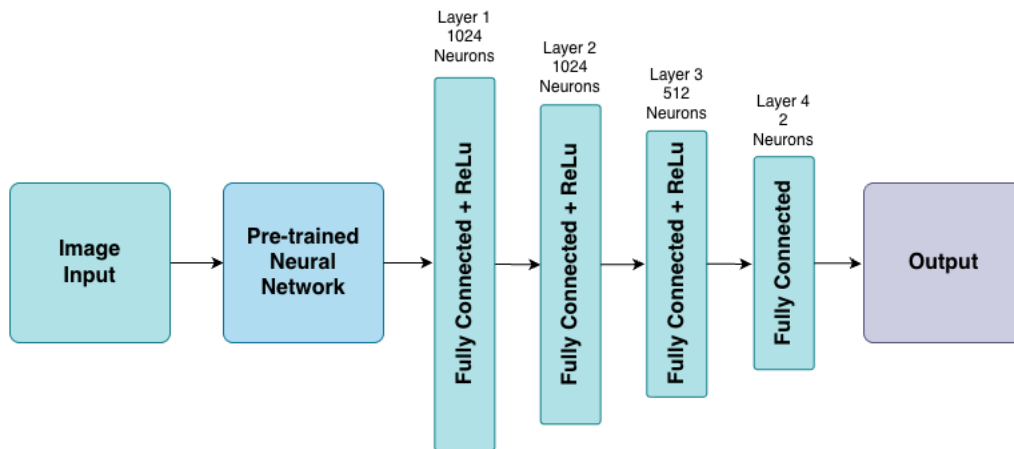


Figure 3.5: Transfer Learning Architecture.

3.2.3.3 Deep learning models

Deep learning is a type of machine learning that teaches artificial neural networks to find patterns and connections in large, complicated datasets. These networks are made up of layers of neurons that are connected to each other. Together, they learn how to get useful information from raw data. Deep learning has made big strides in image classification, object detection, natural language processing, and medical diagnostics because it can capture non-linear features that older models can't. In this study, deep learning models are the main part of the system that classifies coconut diseases. They help the system learn small differences in disease symptoms across leaf images. This part talks about the deep learning models that were used in the research and why they are good for analyzing agricultural images with high accuracy.

3.2.3.3.1 DenseNet121

DenseNet121 is a convolution-based architecture that uses dense skip connections to link each layer to every other layer. This design lets feature maps move smoothly through the network, which speeds up gradient propagation and lowers the chance of gradients disappearing during training. DenseNet121 encourages feature reuse, which means that representations learned earlier can be used by deeper layers. This makes the model smaller and has fewer parameters than other deep architectures. DenseNet121 is especially good for tasks like finding coconut disease because it can keep small details in leaf patterns, textures, and lesion shapes. DenseNet121 is a good backbone model for many agricultural image classification problems because it can generalize well and is efficient.

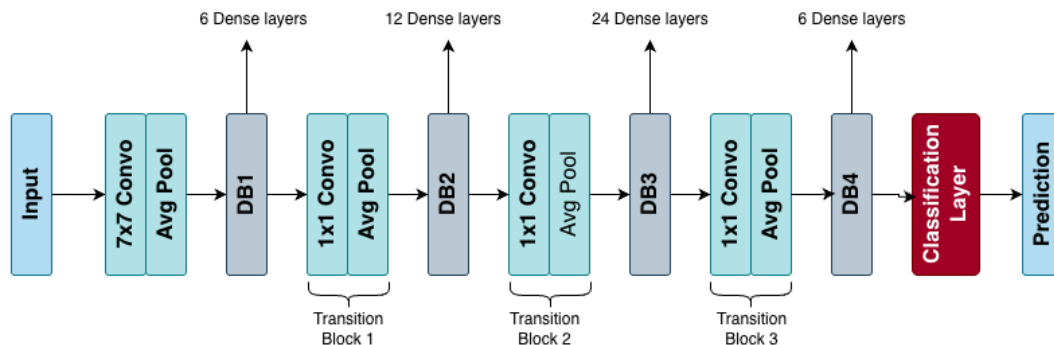


Figure 3.6: DenseNet121 Architecture.

3.2.3.3.2 ResNet50

ResNet50 is a very important deep learning architecture that is often used for image classification. It added residual connections, which help the network deal with the degradation problem by letting gradients go straight through shortcut paths. These residual blocks let you train deeper networks without losing performance. This makes ResNet50 very good at finding both low-level and high-level patterns in images. The architecture strikes a good balance between accuracy and cost, which makes it a good choice for detecting diseases in plants where the model has to pick up on subtle changes in leaf color, texture, and shape. ResNet50 was chosen as one of the benchmark CNN models for this study because it has been shown to work well for classifying plant diseases and can handle images with a wide range of conditions.

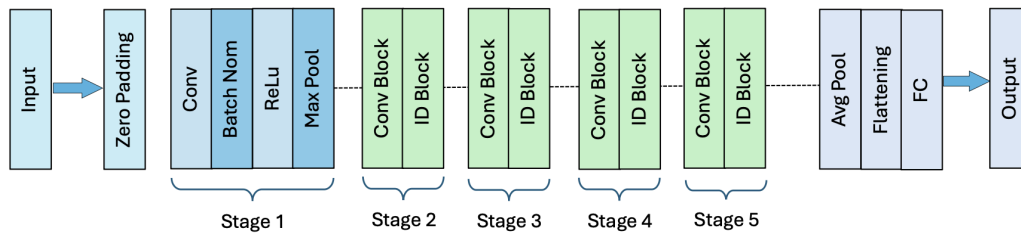


Figure 3.7: ResNet50 Architecture.

3.2.3.3.3 MobileNetV2

MobileNetV2 is a lightweight deep learning architecture that works well on mobile devices and in environments with few resources. It uses inverted residual blocks and depthwise separable convolutions to cut down on computation while keeping accuracy high. The linear layers cut down on changes that aren't needed, while the bottleneck layers keep important feature information. MobileNetV2 is good for real-time farming apps because it works well on devices with limited memory and processing power, like smartphones and edge devices. MobileNetV2 is a good choice for field deployment where computing resources may be limited because it supports fast inference and low power consumption for coconut disease detection.

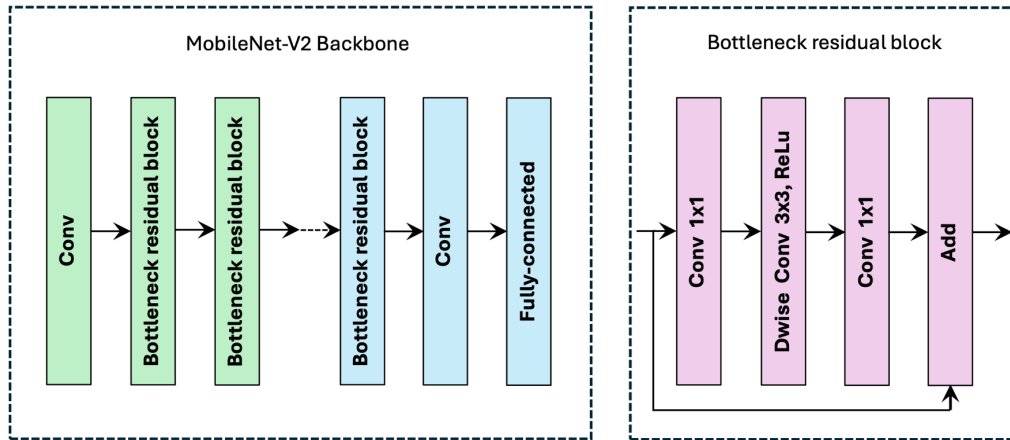


Figure 3.8: MobileNetV2 Architecture.

3.2.3.3.4 MobileNetV3

MobileNetV3 is based on MobileNetV2, but it also adds neural architecture search (NAS) and squeeze-and-excitation (SE) modules to automatically improve the network. There are two sizes of it: Small and Large. Both are made to be as efficient and accurate as possible. MobileNetV3 is even more efficient at using computer resources than the previous version, and it works better on a number of benchmark datasets. This makes it perfect for making portable disease detection tools that can work on mobile phones or embedded boards. MobileNetV3 is a great choice for real-time coconut disease classification tasks in environments with limited resources because it has a lightweight architecture, SE attention mechanisms, and optimized convolutional blocks.

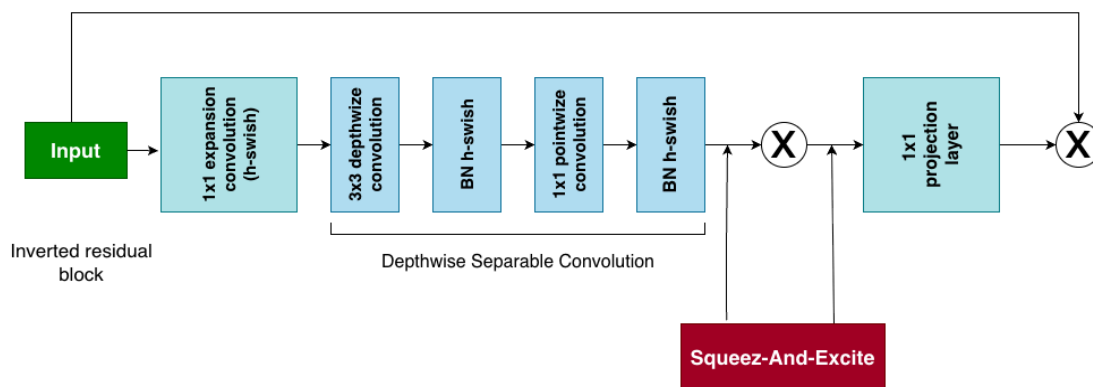


Figure 3.9: MobileNetV3 Architecture.

3.2.3.3.5 Xception

Xception is a CNN architecture that uses depthwise separable convolutions instead of regular convolutions. This helps the model learn depth and channel-wise relationships better. It builds on the idea of Inception modules by completely separating the extraction of spatial and cross-channel features. This makes the architecture more powerful and efficient. Xception is good at complicated visual tasks and is known for being able to pick up on small changes in texture and patterns. This is an important skill for telling the difference between similar coconut disease symptoms. Because the model works well and is easy to build, it is a good starting point for comparing modern CNNs with transformer-based methods for classifying agricultural images.

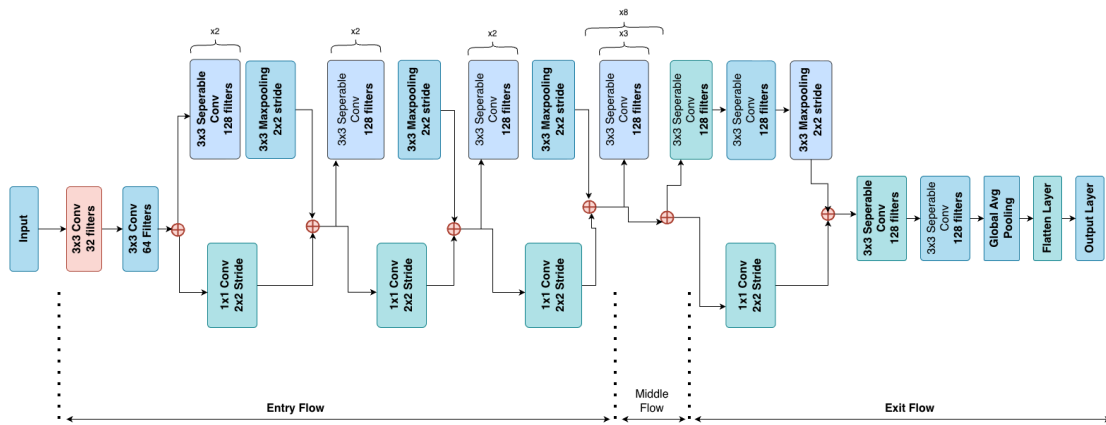


Figure 3.10: Xception Architecture.

3.2.4 Experiment

In this phase, we designed and ran tests to see how well different deep learning architectures worked on the curated coconut disease dataset. The study examined both heavy and lightweight models to comprehend the balance among accuracy, computational duration, and deployment viability. To make sure the comparison was fair, all models were trained on the same preprocessed dataset and in the same experimental conditions. The experiment also involved fine-tuning pretrained weights, using transfer learning, and measuring the model's performance on a number of different metrics.

3.2.4.1 Heavy Model vs Lightweight Models

The tests looked at both heavy models like DenseNet121, ResNet50, and Xception and light models like MobileNetV2 and MobileNetV3. Heavy models usually have more parameters and deeper feature extraction layers, which lets them pick up on subtle disease-related

changes and complex texture patterns. But they need more computing power, take longer to train, and are less useful for mobile or edge devices. On the other hand, lightweight models are better for field applications because they can be used in real time, use less memory, and make decisions faster. This comparison helped us find the best balance between performance and efficiency for real-world farming use.

3.2.4.2 Training and Fine-Tuning

All models were trained using transfer learning, beginning with ImageNet weights that had already been trained. The base layers were frozen at first to keep general feature representations, and the top layers were fine-tuned on the coconut disease dataset to learn patterns that are specific to the domain. Through experimentation, we were able to find the best values for the learning rate, batch size, and number of epochs. Fine-tuning helped each architecture work better with the five disease categories, which made them more accurate and stopped them from overfitting. To improve generalization during training, data augmentation was used, and early stopping was used to save time and effort.

3.2.5 Model Evaluation

To get a full picture of how well the model worked, we used a number of different metrics to test its performance. Accuracy and loss showed how well the model trained in general, while precision, recall, and F1-score showed how well it predicted classes. We used other tools, like confusion matrices and line graphs, to understand how errors were spread out and how training was going. These metrics together made sure that the models were fully tested for reliability, robustness, and consistency.

3.2.5.1 Accuracy

Accuracy looked at the percentage of images that were correctly predicted in all classes. It gave a broad picture of how well the model was working overall. High accuracy showed that the model was good at telling the difference between the five types of coconut disease, while low accuracy meant that the model was either misclassifying or not learning enough.

3.2.5.2 Loss

Loss measured how wrong the model was during training. It helped keep track of how well the network used gradient updates to reduce misclassification. A loss curve that went down showed stable learning, while loss values that changed or were too high showed overfitting or bad optimization.

3.2.5.3 Precision

Precision measured how well the model found a certain disease class without including false positives. The model's high precision meant that it was sure and correct in its predictions for each disease category, which made it useful for avoiding misdiagnosis.

3.2.5.4 Recall

Recall tested how well the model could find all real positive cases in a disease class. High recall values showed that the model could find even small signs of illness, which lowered the chance of missing a diagnosis in real life.

3.2.5.5 F1-Score

The F1-score combined precision and recall into one number, which gave a fair picture of how well a model predicted. It was especially helpful for datasets that weren't balanced because it made sure that both under-prediction and over-prediction were taken into account when judging a model.

3.2.5.6 Confusion Matrix

The confusion matrix showed true positives, false positives, false negatives, and true negatives for each disease category. This gave us a lot of information about predictions for each class. It helped find out which diseases were most often misclassified and gave suggestions for how to improve preprocessing or model tuning.

3.2.5.7 Line Graph (Accuracy and Loss)

To see how learning changed over time, line graphs were made for training and validation accuracy and loss. These graphs showed if the model converged smoothly, overfit the data, or needed more tuning. They made it clear how stable the training was and how the overall performance was improving.

3.2.6 Final Evaluation and Comparison

After training all the models, a final test looked at their accuracy, speed, size, and speed of inference. Heavy models usually had better accuracy but needed more resources. On the other hand, lightweight models had similar performance but were much more efficient. This comparison made it possible to choose a balanced architecture that would work well in real-time systems for detecting coconut diseases. The final results showed that lightweight models like MobileNetV3 worked well while still being very accurate at

classifying things.

3.2.9 Explainable AI (Grad-CAM)

Explainable AI methods help us understand how deep learning models come to their decisions. One of the most popular ways to make visual explanations is with Gradient-weighted Class Activation Mapping (Grad-CAM), which shows the parts of an image that have the most effect on the model's prediction. Grad-CAM is especially useful for finding diseases in crops because it lets experts and end-users check if the model is looking at real disease symptoms and not just noise or background elements. Adding Grad-CAM to this study helps with transparency, trust, and understanding by letting people see how the model works. It also helps find model errors and make the system more reliable when it is used in real-world farming situations.

3.3 Project Plan

This project is completed across several structured phases. Each phase outlines the major tasks and goals required to develop the coconut leaf disease detection system.

Phase 1: Literature Review & Research Setup

- I. Check out current ways to find coconut diseases that use CNN, MobileNet, ResNet, DenseNet, Xception, and transformer models.
- II. Check out current ways to find coconut diseases that use CNN, MobileNet, ResNet, DenseNet, Xception, and transformer models.
- III. Examine methodologies from analogous agricultural disease detection systems to delineate appropriate model architectures for the project.
- IV. Fill in the research gaps and set the direction for the project.

Phase 2: Data Collection & Preprocessing

- I. Get pictures of coconut leaf disease from a number of different online sources.
- II. Get rid of images that are blurry, low-quality, or not related to the topic.
- III. Choose five final classes: Leaf Rot, Coconut Caterpillar, Bud Rot, Stem Bleeding, and Yellow Leaf Disease.
- IV.
- V. Change the size of all the pictures to 224 by 224 pixels.
- VI. Normalize pixel values to make sure that all models get the same input.
- VII. To cut down on class bias, balance the dataset.
- VIII. Use translation, rotation, shearing, scaling, flipping, and contrast adjustment to add to the image.

- IX. Divide the dataset into 20% testing and 80% training.

Phase 3: Model Development & Experimentation

- I. Use transfer learning models like DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3.
- II. Try out both heavy and light architectures in experiments.
- III. Make sure that all models go through the same training process.
- IV. Improve the learning rate, batch size, choice of optimizer, dropout, and regularization.
- V. Keep an eye on how well all five disease classes are doing.

Phase 4: Model Optimization & Fine-Tuning

- I. Do a lot of fine-tuning on the pre-trained layers.
- II. Use early stopping to stop overfitting.
- III. To make things more general, use dropout and regularization.
- IV. Change the hyperparameters to make the model more accurate and cut down on training time.
- V. Make it easier for lightweight models like MobileNetV2 and MobileNetV3 to work in real time.

Phase 5: Model Evaluation

- I. Evaluate the models using:
 - A. Accuracy
 - B. Loss
 - C. Precision
 - D. Recall
 - E. F1-score
- II. Generate confusion matrices for class-wise error analysis.
- III. Plot accuracy and loss curves for training/validation comparison.
- IV. Compare heavy vs lightweight model performance.
- V. Apply Grad-CAM to visualize disease-affected leaf regions.

Phase 6: Conclusion & Report Writing

- I. Analyze experimental results and identify the best-performing model.
- II. Summarize comparison between heavy (DenseNet/Xception) and lightweight (MobileNet) models.
- III. Document the entire methodology, evaluation, and outcomes.
- IV. Prepare the final research report and project documentation.

3.3.2 Research Resources

I. Hardware

- A. GPU-enabled environment (Google Colab).
- B. Sufficient RAM and storage for training multiple models.

II. Software

- A. Python programming environment.
- B. TensorFlow, Keras, scikit-learn, NumPy, Matplotlib.
- C. GitHub for version control.

III. Dataset

- A. Coconut leaf disease images collected from multiple online repositories.
- B. Five disease classes after cleaning and selection.

IV. Tools

- A. Google Drive for storage.
- B. Colab notebooks for cloud-based training and visualization.

3.3.3 Risk Management

I. Data Scarcity

- A. Use augmentation techniques to expand image variety.
- B. Combine multiple sources to improve dataset richness.

II. Overfitting

- A. Implement early stopping and dropout.
- B. Monitor validation performance closely.

III. Hardware Limitations

- A. Optimize training using lightweight models.
- B. Utilize cloud GPU sessions efficiently.

3.3.4 Expected Deliverables

I. Coconut Leaf Disease Classification System:

A deep learning system that can correctly identify five common coconut leaf diseases (Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease) using models like DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. The goal of this system is to help farmers and agricultural workers by quickly and automatically

identifying diseases from pictures of leaves.

II. Comprehensive Research Paper:

A full research paper that talks about how to collect data, clean it up, run experiments, train models, use evaluation metrics, explain Grad-CAM, and compare the final model. The paper shows how lightweight and heavy CNN models can help find coconut diseases and how transfer learning can help classify agricultural images.

III. Well-Structured Codebase:

A repository that is easy to clean and maintain and has scripts for preprocessing, training, fine-tuning, evaluating, Grad-CAM visualization, and comparing models. The codebase will make it easier for future researchers to build on the system and make it possible to reproduce results.

IV. Model Performance Benchmarking:

A complete comparison of several architectures, both heavy (DenseNet121, Xception) and lightweight (MobileNetV2, MobileNetV3), was done using accuracy, precision, recall, F1-score, and confusion matrix. This comparison will help you choose the best models for agricultural apps that work in real time or on mobile devices.

V. Explainable AI Integration (Grad-CAM):

Visual explanation maps that show which parts of coconut leaves are infected. This makes things clearer and helps users understand how the model makes choices. This makes the system more reliable and useful for diagnosing problems in the field.

VI. Practical Agricultural Impact:

A disease detection workflow that can be changed to work on mobile devices or in real time. The system helps reduce crop loss, makes it less necessary to rely on expert inspection, and supports smart agriculture projects that aim to manage coconut disease by automating early-stage detection.

3.4 Task Allocation

This table depicts the timeline of the principal activities in each period of the project, from week 12 to week 48.

[illegible]

Preprocess all the data																			
Model training																			
Create a demo application.																			

3.5 Summary

This chapter provides a comprehensive overview of the research methodology employed for the classification system of coconut leaf disease. The chapter explains how transfer learning was used to classify five major coconut diseases using several deep learning models, such as DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. Each model was trained on a carefully chosen and preprocessed dataset. The images were cleaned, resized to 224×224, normalized, balanced, and augmented by translation, rotation, shearing, scaling, flipping, and changing the contrast. The methodology also showed how both heavy and lightweight architectures can be used to compare performance and computational efficiency. This helped create a system that can be used in real time and on mobile devices. The chapter also went over the whole experimental process, which included splitting the training and testing data, fine-tuning, evaluating the model, and comparing its performance using accuracy, precision, recall, F1-score, confusion matrix, and loss/accuracy line graphs. To make it easier to understand, Explainable AI methods like Grad-CAM were used to show areas of the body that were affected by disease and make sure that predictions were clear. The chapter also talked about the project's overall plan, what resources it would need, how to handle risks, and what the expected deliverables would be. This method creates a full workflow for making a deep learning-based coconut disease detection system that is reliable, scalable, and useful. It can help with early diagnosis and improve modern farming methods.

Chapter 4

Implementation and Results

This chapter describes how the suggested coconut disease classification system was put into action, including setting up the environment, handling the dataset, training the model, and checking the results. It also shows the results of tests done on several deep learning models, including DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. After that, it talks about how well they did on important metrics.

4.1 Environment Setup

4.1.1 Hardware Requirements

When it comes to image-based classification, the hardware setup is very important for managing the training workload of deep learning models. A system with a powerful GPU was used to cut down on training time and do the complicated math that deep architectures like DenseNet121 and Xception need. Even when trying out larger batch sizes or deeper models, GPU acceleration makes sure that model training, backpropagation, and parallel processing all work well. Along with GPU support, enough RAM and processing power were needed to handle data preprocessing and augmentation. To store the raw dataset, preprocessed images, training logs, checkpoints, and visual outputs from experiments, at least 100 GB of space was needed.

4.1.2 Software Requirements

We used Python to set up the software environment because it has a lot of tools for machine learning and deep learning. The implementation was done on Windows. Installing CUDA and cuDNN made sure that TensorFlow and other frameworks could use GPU resources in the best way possible. Python was the main way to build deep learning pipelines, process data, train models, make visualizations, and check performance. This stable software setup made sure that everything went smoothly at every stage of the project.

4.1.3 Frameworks and Libraries

The project depended a lot on modern deep learning frameworks that make it easier to build models. TensorFlow and Keras were the main libraries used to build and train models because they are flexible and come with built-in support for pre-trained architectures like DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3. These frameworks have the best tools for transfer learning, which makes it easier to use big pre-trained networks to classify coconut diseases. We used other Python libraries like NumPy and Pandas to work with and change the data, and Matplotlib and Seaborn to make training curves and model performance visualizations. Scikit-learn was an important part of the evaluation workflow because it helped with tasks like

making classification reports, confusion matrices, and splitting datasets.

4.1.4 Dataset Handling and Preprocessing Tools

The dataset for this project was made up of pictures taken from several online repositories. It focused on five main types of coconut diseases: Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease. Before training the model, all of the images were preprocessed with tools like PIL and OpenCV to make sure they were all the same size (224×224), had the same pixel values, and were all formatted the same way. We used Keras ImageDataGenerator and the Albumentations library to artificially add variety to the dataset and help the models learn strong patterns. Using augmentation methods like rotation, translation, scaling, shearing, flipping, and contrast adjustments helped keep the models from overfitting and get ready for real-world changes. This phase also included label encoding and balancing the dataset so that all classes were equally represented.

4.1.5 Version Control and Collaboration

Git was used to keep track of different versions of the code, which helped keep the workflow clean and organized during development. Commits kept track of every major change to the codebase, such as changes to preprocessing and updates to model parameters. GitHub was the remote repository, which made it easy to access, back up, and work together. Putting the project on GitHub made things clearer, helped keep track of the experiment's history, and made it easier to compare and document models.

4.1.6 Model Training Configuration

Transfer learning was used to train the models. This involved fine-tuning pre-trained architectures like DenseNet121, ResNet50, Xception, MobileNetV2, and MobileNetV3 on the coconut disease dataset. We used ImageNet weights to start these models so they could use features that had already been learned that were useful for image classification. To get stable convergence, training was done with a batch size of 32. We used categorical cross-entropy as the loss function because the task was to classify multiple classes. To keep the distribution across all five disease classes balanced, an 80:20 training and testing split was used. The training setup also had callbacks like early stopping and model checkpointing to stop the model from overfitting and save the one that worked best.

4.1.7 Evaluation Setup

To see how well the trained models worked, we had to calculate a number of important performance metrics that show how well they could classify data. We used scikit-learn tools to figure out the accuracy, precision, recall, and F1-score. These metrics showed how well each model was able to tell the difference between the five types of coconut diseases. To see how well each class did and find possible misclassifications, confusion matrices were made. Line graphs showing the accuracy and loss of training and validation made it easy to see how the model was converging and how stable the training was. This evaluation setup made it possible to do a thorough and reliable comparison of several deep learning architectures.

4.1.8 Working and Communication Tools

Google Colab was the main place where code was written, procedures were documented, results were visualized, and iterative testing was done during the development and testing process. It was easier to improve the preprocessing pipeline, adjust hyperparameters, and look at model outputs in its interactive environment. Google Meet, Zoom, and Telegram were some of the tools that helped with communication and coordination. They made it easier to talk about things, share updates, and keep track of experiment schedules. These tools made it easier for people to work together and helped keep the research process organized.

4.2 Testing and Evaluation/Performance/Comparative Analysis

During the evaluation phase, we trained and tested five deep learning models—DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception—on the coconut disease dataset. This dataset had five classes: Bud Rot, Coconut Caterpillar, Leaf Rot, Stem Bleeding, and Yellow Leaf Disease. All models were trained in the same way, with the same preprocessing, the same augmentation strategies, and an 80–20 split between training and testing. We used accuracy, precision, recall, F1-score, confusion matrices, and loss–accuracy learning curves to measure performance. All of the models were good at classification, but their performance varied when it came to generalization, stability, and computational efficiency. As expected, heavier architectures like DenseNet121 and ResNet50 got scores that were almost perfect. On the other hand, lightweight models like MobileNetV2 and MobileNetV3 performed well but cost much less to run. Based on the results, MobileNetV2 was the best choice for real-time diagnosis and explainability through Grad-CAM because it had the best balance of accuracy, efficiency, and robustness.

4.2.1 Training and Validation Curve Analysis (Accuracy & Loss)

4.2.1.1 MobileNetV2

MobileNetV2 showed very stable performance, with training accuracy of 99.8% and validation accuracy of 98.7%. The training loss went down to 0.1%, and the validation loss stayed at 0.4%, which is a sign of great generalization. The curves were smooth and didn't move up and down, which showed that MobileNetV2 is strong even though it is light.

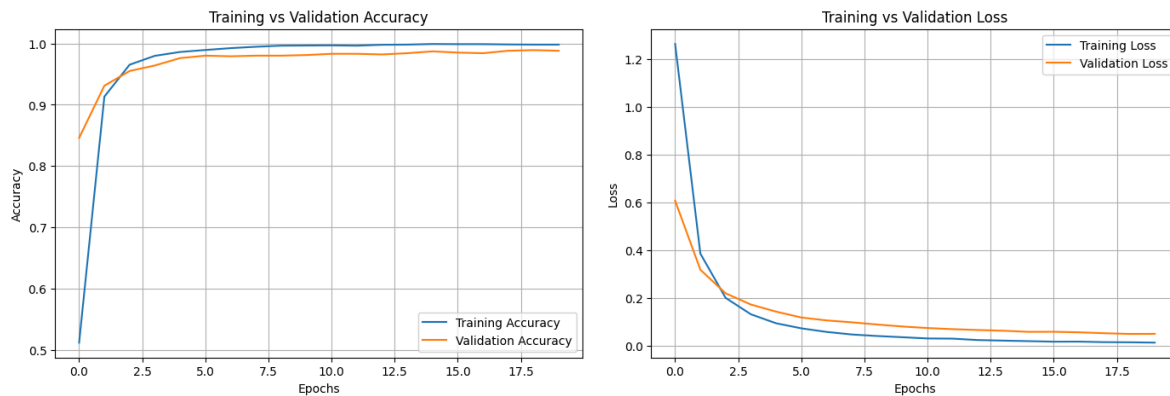


Figure 4.1: MobileNetV2 Accuracy and Loss

4.2.1.2 MobileNetV3

MobileNetV3 also developed quickly, with a training accuracy of 99.82% and a validation accuracy of 97.60%. The loss during training went down to 0.1%, and the loss during validation went up to 0.6%. The curves were mostly stable, but the fact that the gap between training and validation loss was bigger than MobileNetV2's shows that generalization performance was a little worse.

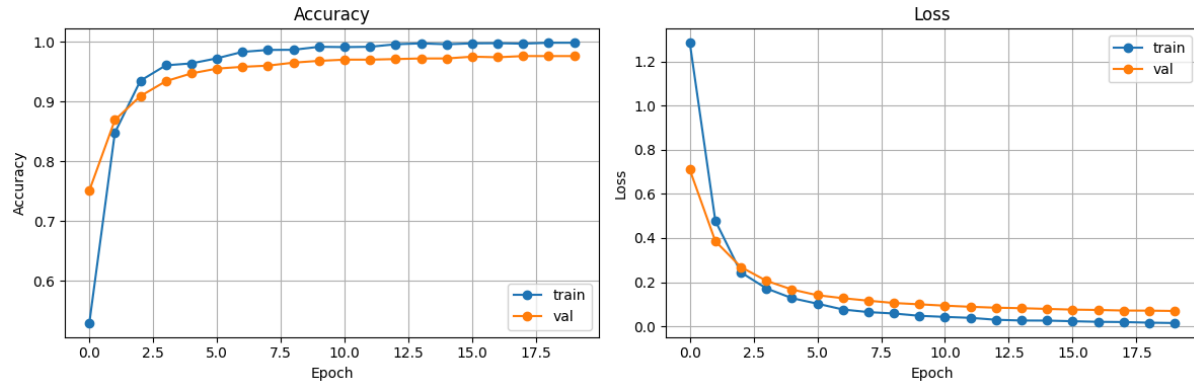


Figure 4.2: MobileNetV3 Accuracy and Loss

4.2.1.3 DenseNet121

The training and validation accuracy curves for DenseNet121 went up quickly and met at 100% and 99.99%, which shows that the model learned very well from the start. Both losses were close to zero, which showed that the model found disease patterns that were different from each other without overfitting. The small difference between the training and validation curves shows that the model generalizes well, which is in line with the way DenseNet121 connects things.

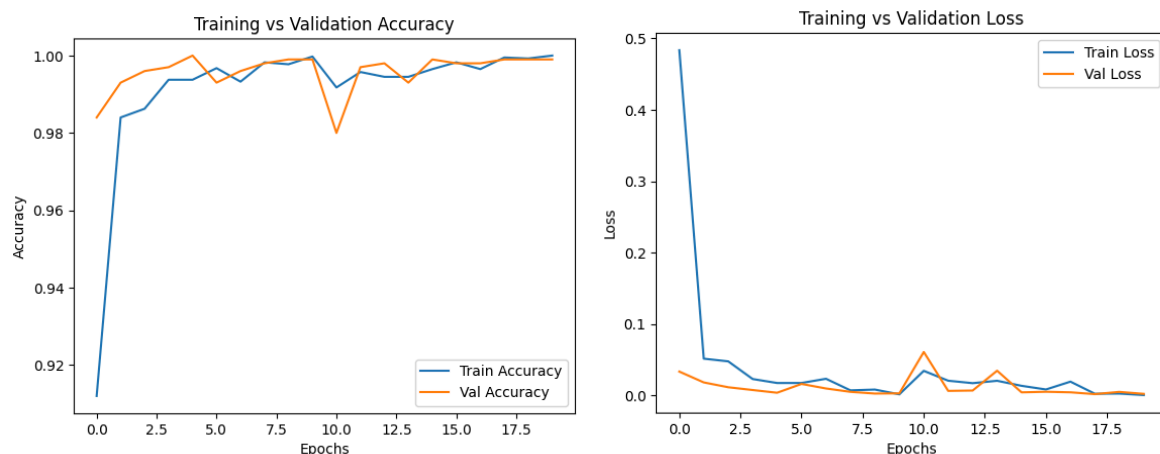


Figure 4.3: DenseNet121 Accuracy and Loss

4.2.1.4 ResNet50

ResNet50 acted in a similar way, with training accuracy reaching 100% and validation accuracy staying at 99.90%. The loss curves kept going down and were very close to each other, which showed that residual connections worked well to stop gradient degradation. There was no divergence between the training and validation curves, so the model converged smoothly.

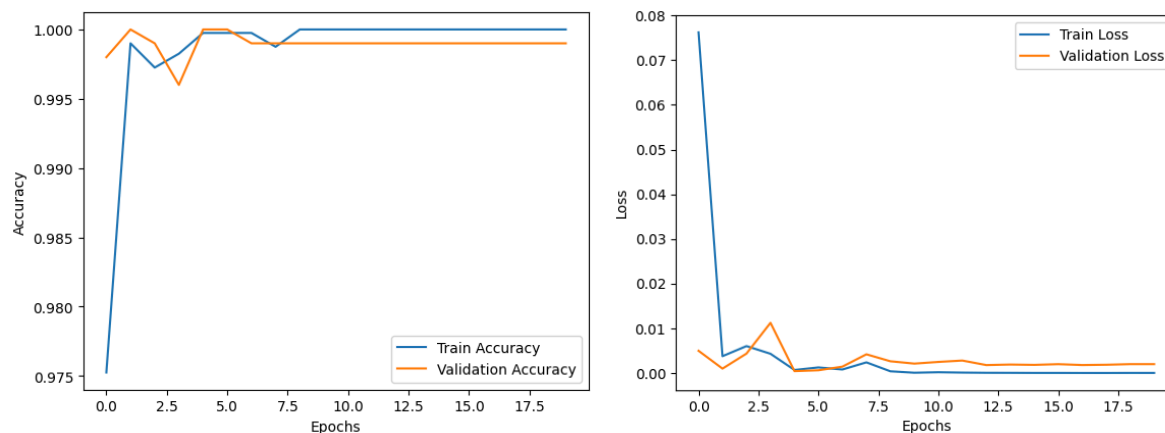


Figure 4.4: ResNet50 Accuracy and Loss

4.2.1.5 Xception

Both curves for Xception kept getting better, with training accuracy reaching 98.22% and validation accuracy reaching 98.69%. The loss curves were smooth, with a training loss of 0.07% and a validation loss of 0.08%, which showed that Xception generalized well. But the model needed more computing power to get to this point.

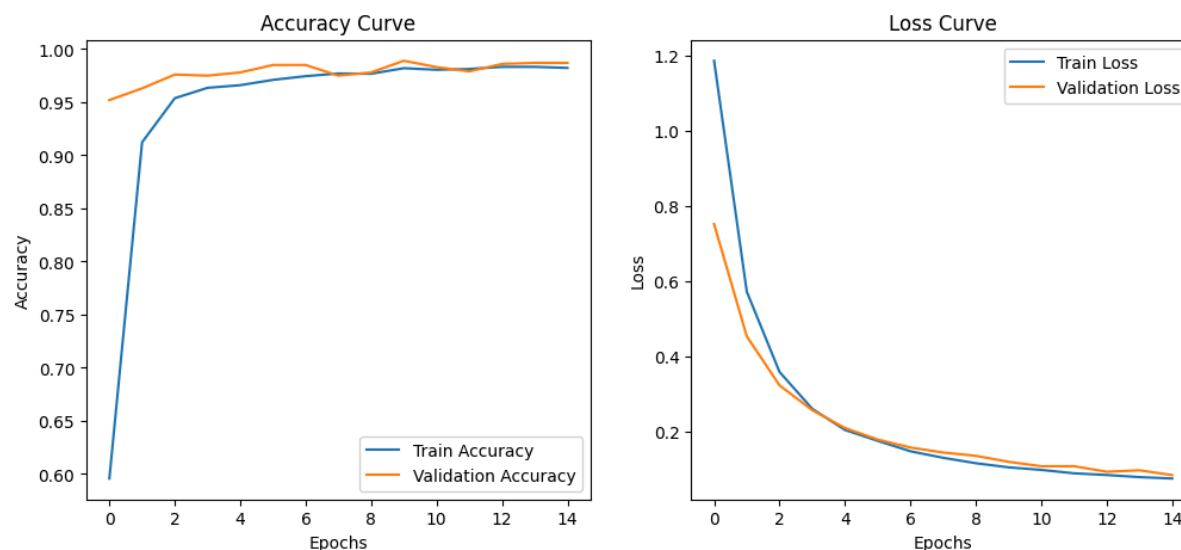


Figure 4.5: Xception Accuracy and Loss

4.2.2 Evaluation Metrics Analysis (Precision, Recall, F1-Score)

4.2.2.1 MobileNetV2

MobileNetV2 got almost perfect scores on the tests: Precision = 0.99, Recall = 0.99, and F1-Score = 0.99. These results show that the model works well for each class and can be used in a wide range of situations. The model worked just as well as heavy architectures, but it didn't cost much to run.

Table 4.1: Classification Report of MobileNetV2

Class	Precision	Recall	F1-Score	Support
Bud_Rot	0.96	1.00	0.98	200
Coconut_Caterpillar	1.00	1.00	1.00	200
Leaf_Rot	1.00	0.94	0.97	200
Stem_Bleeding	1.00	1.00	1.00	200
Yellow_Leaf_Disease	0.99	1.00	0.99	200
Accuracy			0.99	1000
Macro avg	0.99	0.99	0.99	1000
Weighted avg	0.99	0.99	0.99	1000

4.2.2.2 MobileNetV3

MobileNetV3 also did well, but not quite as well as MobileNetV2. Its Precision, Recall, and F1-Score were all 0.98. The small drop is due to some disease types being incorrectly classified because their patterns overlap.

Table 4.2: Classification Report of MobileNetV3

Class	Precision	Recall	F1-Score	Support
Bud_Rot	1.00	1.00	1.00	200
Coconut_Caterpillar	0.96	1.00	0.98	200
Leaf_Rot	0.99	0.93	0.96	200
Stem_Bleeding	1.00	0.95	0.97	200
Yellow_Leaf_Disease	0.93	1.00	0.97	200
Accuracy			0.98	1000
Macro avg	0.98	0.98	0.98	1000
Weighted avg	0.98	0.98	0.98	1000

4.2.2.3 DenseNet121

DenseNet121 got perfect scores on all of the tests, with Precision = 1.00, Recall = 1.00, and F1-Score = 1.00. This means that the model can perfectly classify all five disease classes, which shows how well it can extract features and how stable it is.

Table 4.3: Classification Report of DenseNet121

Class	Precision	Recall	F1-Score	Support
Bud_Rot	0.99	1.00	1.00	189
Coconut_Caterpillar	1.00	1.00	1.00	191
Leaf_Rot	1.00	1.00	1.00	213
Stem_Bleeding	1.00	1.00	1.00	204
Yellow_Leaf_Disease	1.00	1.00	1.00	203
Accuracy	1.00	1.00	1.00	1000
Macro avg	1.00	1.00	1.00	1000
Weighted avg	1.00	1.00	1.00	1000

4.2.2.4 ResNet50

ResNet50 got the same results as DenseNet121, with Precision = 1.00, Recall = 1.00, and F1-Score = 1.00. This shows that ResNet50 did a great job of finding small differences between diseases and correctly classifying every test sample.

Table 4.4: Classification Report of ResNet50

Class	Precision	Recall	F1-Score	Support
Bud_Rot	1.00	1.00	1.00	189
Coconut_Caterpillar	1.00	0.99	1.00	191
Leaf_Rot	1.00	1.00	1.00	213
Stem_Bleeding	1.00	1.00	1.00	204
Yellow_Leaf_Disease	1.00	1.00	1.00	203
Accuracy	1.00	1.00	1.00	1000

Macro avg	1.00	1.00	1.00	1000
Weighted avg	1.00	1.00	1.00	1000

4.2.2.5 Xception

Xception got high scores on evaluations, which is in line with its accuracy in validation. With a validation accuracy of 98.69%, its precision, recall, and F1-score showed that it could find small disease features. But it didn't do better than DenseNet121 or ResNet50.

4.2.3 Confusion Matrix Analysis

4.2.3.1 MobileNetV2

MobileNetV2 had the cleanest confusion matrix of all the lightweight models, with only a few samples that were incorrectly classified. True positives were always high in all classes. The few mistakes that were found were small and didn't have a big effect on overall performance. This strong consistency made MobileNetV2 even more of a lightweight model.

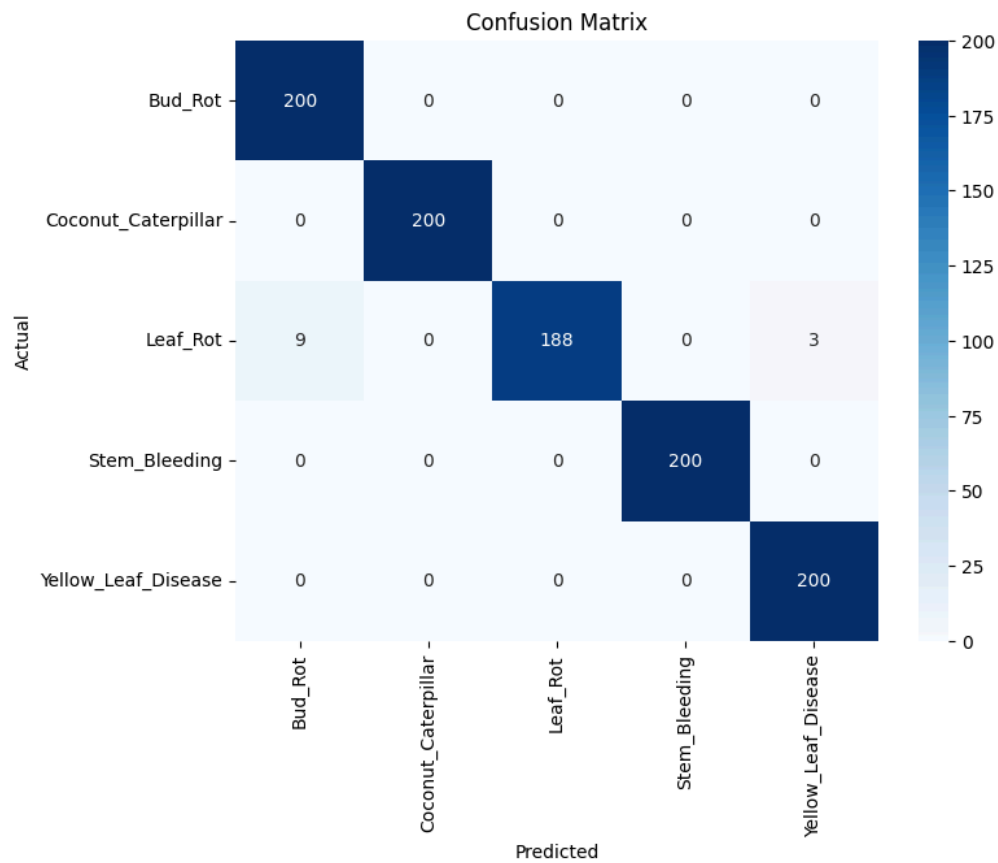


Figure 4.6: Confusion Matrix MobileNetV2

4.2.3.2 MobileNetV3

MobileNetV3 had a few more misclassifications than MobileNetV2. Most of the mistakes happened between diseases that looked similar, like Leaf Rot and Yellow Leaf Disease. The model still had a high overall accuracy, though.

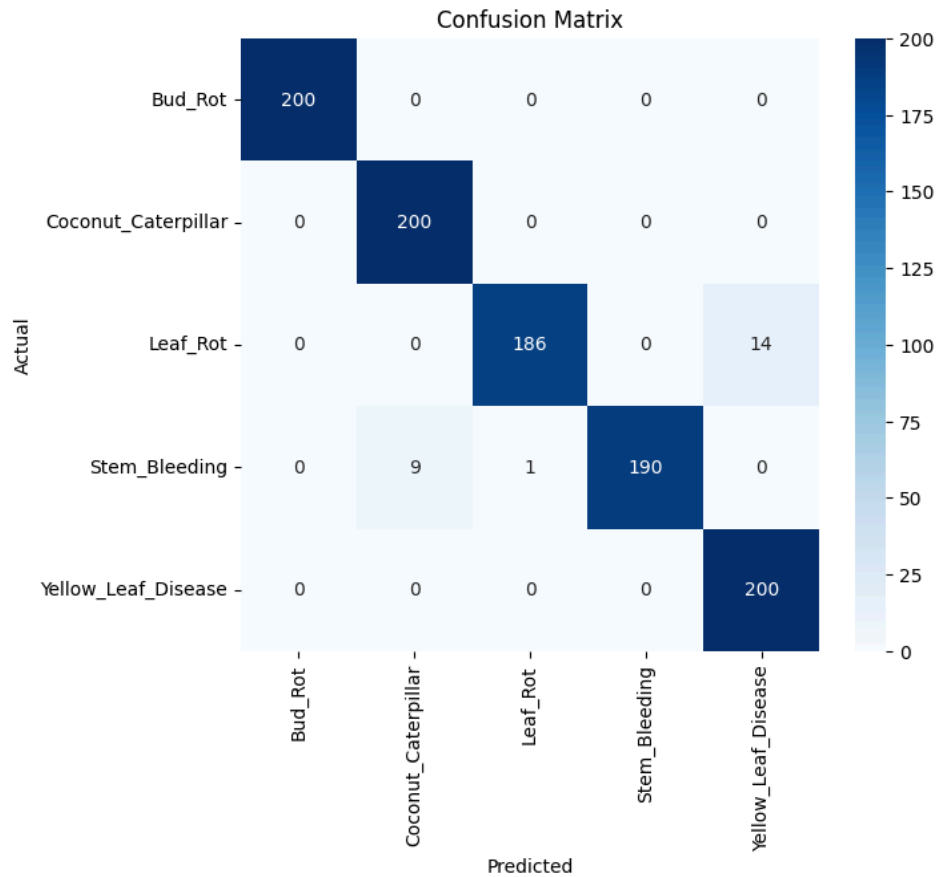


Figure 4.7: Confusion Matrix MobileNetV3

4.2.3.3 DenseNet121

The confusion matrix for DenseNet121 showed that all five classes were perfectly classified, with every sample being predicted correctly. The diagonal values were the highest, and there were only 1 wrong classification, which shows that the features were very good at telling the difference between them.

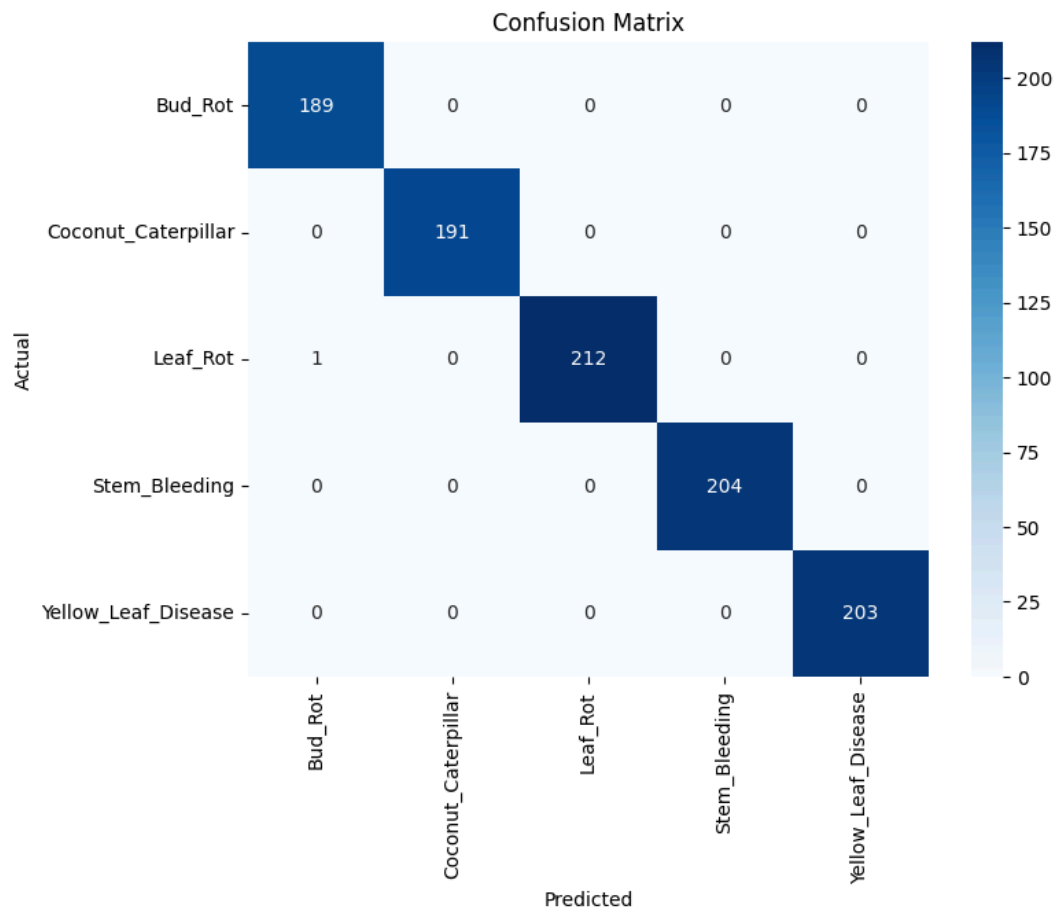


Figure 4.8: Confusion Matrix DenseNet121

4.2.3.4 ResNet50

ResNet50 also made perfect predictions, just like DenseNet121. Every class got 100% correct except 1 misclassification between Coconut Caterpillar and Stem Bleeding, which shows that the model can effectively extract structural features.

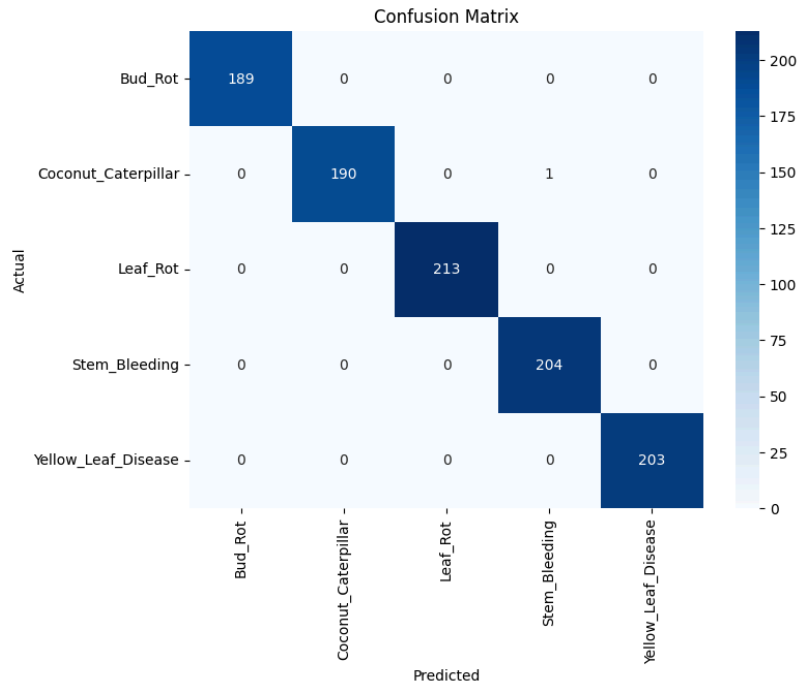


Figure 4.9: Confusion Matrix ResNet50

4.2.3.5 Xception

The confusion matrix for Xception showed that it was good at classifying things, but it made more mistakes than MobileNet models. This shows that Xception learned complex features well, but its deeper architecture sometimes made it too good at fitting.

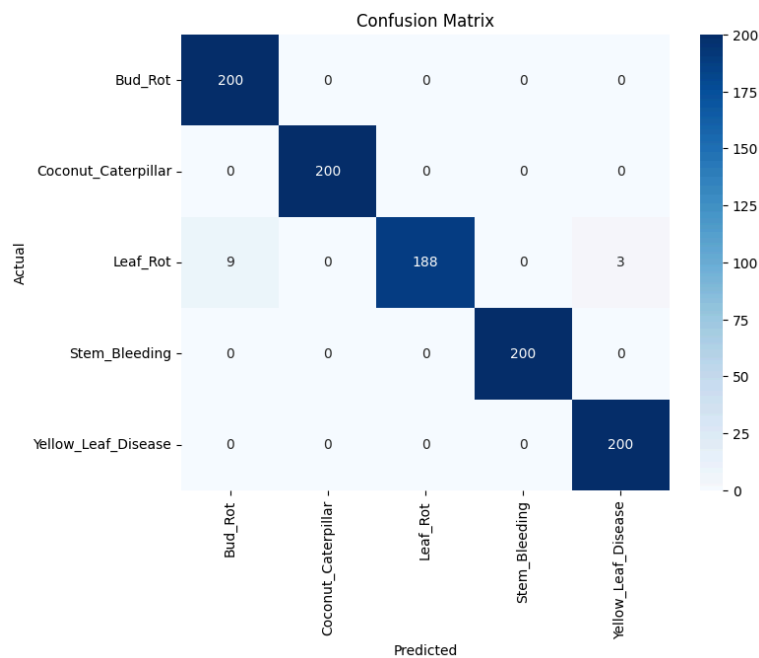


Figure 4.10: Confusion Matrix Xception.

4.2.4 Comparative Analysis of All Models

In terms of raw performance, DenseNet121 and ResNet50 were the best models because they had perfect evaluation metrics. But these models are hard on computers, take longer to train, and can't be used in mobile or real-time apps. Xception also did a great job, but it needed a lot of computing power. MobileNetV2, on the other hand, had a great balance of high accuracy (98.79% validation accuracy) and low complexity, as well as strong metric scores (0.99 Precision, Recall, and F1-Score). The model's confusion matrix showed very few mistakes, and the learning curves were stable and the model converged quickly. MobileNetV3 also did well, but it didn't generalize as well as MobileNetV2. We chose MobileNetV2 as the best model because it was the most efficient, accurate, stable for generalization, and suitable for use in the field. We then used Grad-CAM explainability on it to show disease-specific activation regions for transparency and trustworthiness.

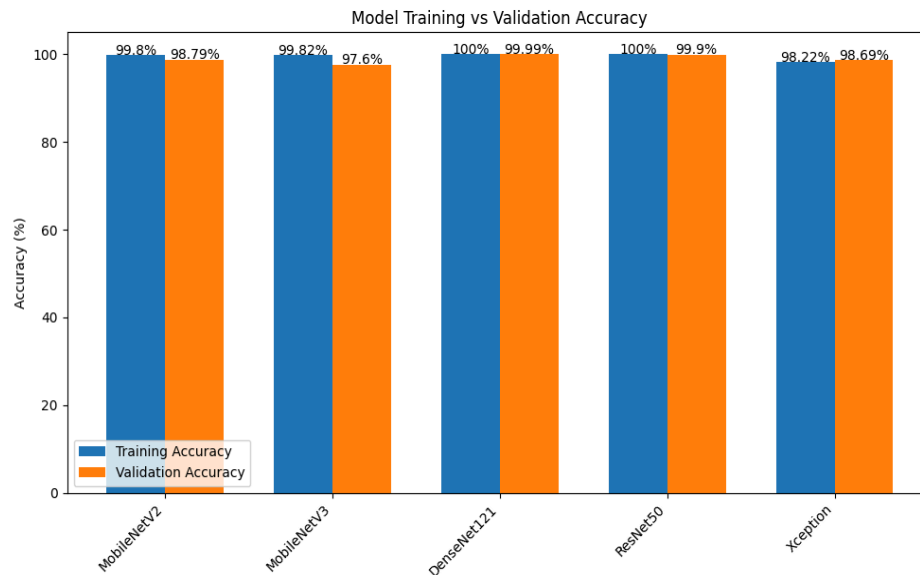


Figure 4.11: Comparison of accuracies between all models.

Figure 4.11 shows a bar chart that compares the training and validation accuracies of five deep-learning architectures: MobileNetV2, MobileNetV3, DenseNet121, ResNet50, and Xception. DenseNet121 and ResNet50 were the best-performing models, with both achieving 100% training accuracy and validation accuracies of 99.99% and 99.90%, respectively. This shows that they can generalize very well. MobileNetV2 did well too, with a training accuracy of 99.80% and a validation accuracy of 98.70%. MobileNetV3 did even better, with a training accuracy of 99.82% and a validation accuracy of 97.60%. This shows that the two models are well-balanced in terms of performance and cost. Xception did a little worse than the other models, but it still did well, finishing with 98.22% training accuracy and 98.69% validation accuracy. DenseNet121 and ResNet50 were the most accurate models overall. MobileNet variants had high accuracy and

were also computationally efficient. Xception had consistent but slightly lower accuracy.

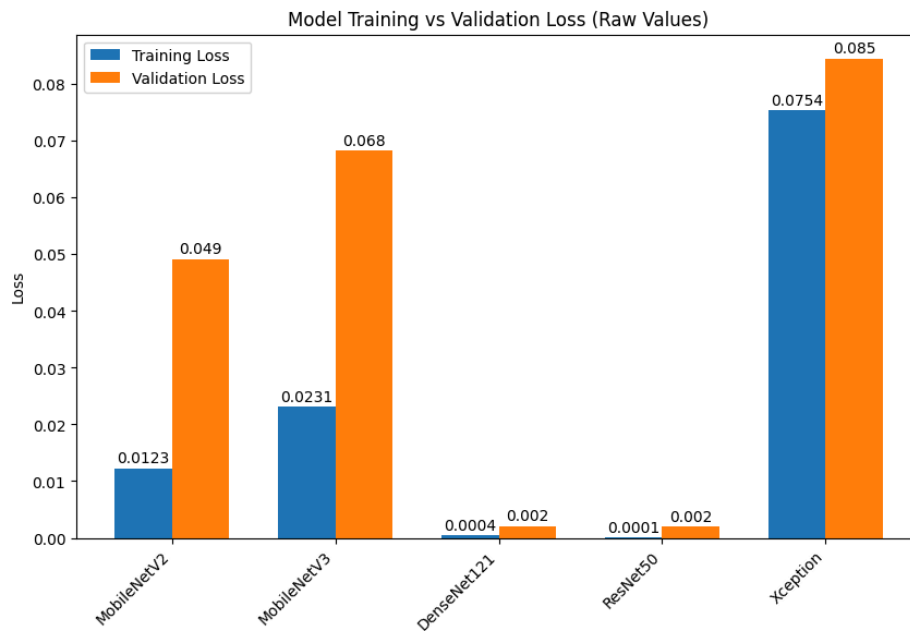


Figure 4.12: Comparison of losses between all models.

Figure 4.12 shows a bar chart that compares the final training and validation losses for the five models. DenseNet121 and ResNet50 showed better convergence behavior, with training losses of 0.0004 and 0.0000 and validation losses of 0.0021 and 0.0020, respectively. This means that the models made very few mistakes when making predictions. MobileNetV2 had a training loss of 0.0123 and a validation loss of 0.0491. MobileNetV3 had a training loss of 0.0231 and a validation loss of 0.0682. Both showed stable learning patterns that would work well for lightweight deployment. Xception had the highest training and validation losses, 0.0754 and 0.0845, respectively. This means that it made bigger mistakes when making predictions. Overall, DenseNet121 and ResNet50 did the best job of reducing errors, MobileNet models had moderate loss values but were still very efficient, and Xception had higher loss but still worked well.

4.2.5: Explainable AI Implementation

The study used explainable AI (XAI) to make the pest classification models more clear and easy to understand, so that both technical and non-technical users can understand how decisions are made. XAI can help check if a deep learning model is looking at the right parts of an image before making a prediction because deep learning models often work like black boxes. This is especially important in farming, where getting the wrong information could hurt crops and make it harder for farmers to make decisions. To do this, Grad-CAM (Gradient-weighted Class Activation Mapping) was used on the best-performing lightweight model, MobileNetV2, which was chosen because it was very accurate and worked well in real-time, resource-limited settings. Grad-CAM

makes heatmaps that show the parts of an image that had the biggest impact on the final classification. By putting these heatmaps on top of the original pest images, the system shows how the model predicts leaf textures, color changes, pest marks, and infected areas.

The visual explanations showed that MobileNetV2 always focused on features that were biologically important and meaningful, like pest lesions, discoloration, and damaged tissue. This validation gives us confidence that the model is learning the right patterns and not just relying on random background noise or cues. Adding XAI not only makes the model more reliable, but it also builds trust in users. This helps farmers and agricultural experts better understand why a certain pest category was chosen. This step makes sure that the technology is used responsibly, lowers the chances of misunderstanding, and helps people make better and more confident decisions in real-world farming situations.

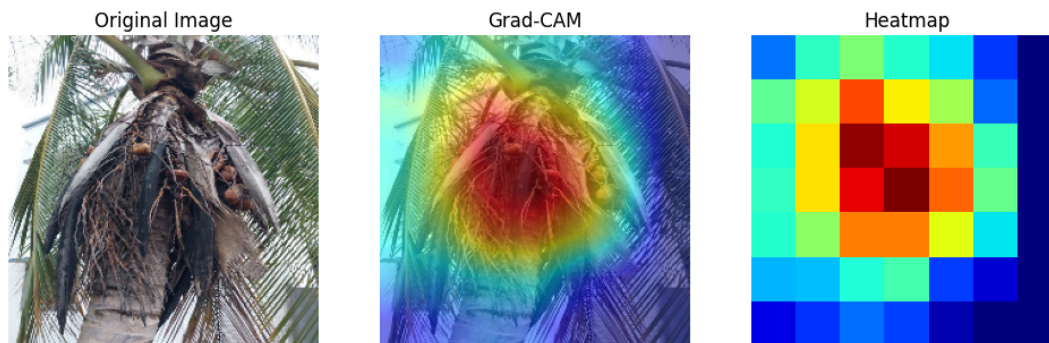


Figure 4.13: Comparison of losses between all models.

4.3 Results and Discussion

The experimental results show that all five deep learning models did very well on the coconut disease dataset, with accuracy levels over 97% for all of them. Heavyweight architectures like DenseNet121 and ResNet50 got perfect or almost perfect results, with 100% training accuracy and almost 100% validation accuracy, thanks to perfect precision, recall, and F1-scores. This shows that they can find very unique features even in diseases that look similar, like Leaf Rot and Yellow Leaf Disease. But these models need more computing power and longer training times, which makes them less useful for on-device or real-time applications. MobileNetV2 and MobileNetV3, which are lightweight models, got competitive results while costing a lot less to run. MobileNetV2 was the best lightweight model, with a training accuracy of 99.8% and a validation accuracy of 98.7%. Its evaluation metrics were always 0.99 precision, 0.99 recall, and 0.99 F1-score. Its confusion matrix shows that only a few samples were misclassified, which means that MobileNetV2 works well for all five disease categories. MobileNetV3 also did well, with a

validation accuracy of 97.60%. However, it had a slightly higher rate of misclassification than MobileNetV2. Xception did very well, with a validation accuracy of 98.69% and stable accuracy and loss curves that showed good generalization. MobileNetV2 was the best model overall because it was the most accurate, generalizable, and efficient in terms of computing power. It was perfect for use in agriculture because it could capture important disease features while keeping inference time low. Because of this benefit and its strong and consistent evaluation metrics, MobileNetV2 was chosen for Grad-CAM implementation. The visualization clearly showed which parts of the coconut leaves were affected by the disease, which proved that the model's choices were understandable and matched the real symptoms. The results show that MobileNetV2 is not only dependable but also useful for early detection systems that work in the field, especially when there aren't many computing resources available.

4.4 Summary

This chapter discussed how to use, test, and evaluate the performance of several deep learning models for classifying coconut diseases. To make sure the results could be repeated, the environment setup, preprocessing pipeline, and training settings were all explained. Five models MobileNetV2, MobileNetV3, DenseNet121, ResNet50, and Xception were trained and tested using the same methods. Their performance was measured using accuracy, loss curves, precision, recall, F1-score, and confusion matrices. The experimental results showed that all of the models worked well. DenseNet121 and ResNet50 got almost perfect scores, and the lightweight models were very efficient. MobileNetV2 was the most balanced model in terms of accuracy and cost of computation, which made it a good choice for real-time use. The Grad-CAM implementation confirmed the interpretability of the chosen model, providing visual insights into how the model detects disease-specific areas. The chapter ends by saying that deep learning is very good at finding coconut diseases early and accurately, and that lightweight architectures are useful for field deployment.

Chapter 5

Engineering Standards and Design Challenges

This chapter provides an overview of the engineering standards, practices, and design factors that went into making the coconut disease classification system. It talks about the software, hardware, and communication standards that were used throughout the project. It also talks about the biggest design problems that came up when working with deep learning models, image datasets, and deployment that used fewer resources. The chapter ends with analyses of the system's long-term effects and feasibility in terms of society, the environment, sustainability, ethics, and finances.

5.1 Compliance with the Standards

The coconut disease detection system was built using best practices in deep learning, computer vision, dataset preprocessing, and model optimization. The dataset was meticulously curated through uniform resizing, normalization, and augmentation techniques to align with the input requirements of all CNN architectures employed in the study, including DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception. These preprocessing steps made sure that images with different lighting, resolutions, and backgrounds were all the same before they got to the models. The evaluation process used standard performance metrics like accuracy, precision, recall, F1-score, and training-validation loss curves, which made it easy to compare all the models. The training setups followed best practices, like using early stopping and controlled changes to the learning rate to keep from overfitting. The project also followed ethical and operational standards by making sure that the dataset was available to everyone and didn't contain any private information. MobileNetV2 was chosen as the final deployment-ready model because it met the industry's need for a balance between accuracy and computational efficiency, especially for real-world applications that rely on mobile devices. We used augmentation, regularization, transfer learning, and choosing architectures that were both fast and accurate to solve problems like limited dataset diversity, visual similarities between disease categories, class imbalance, and model complexity. Overall, the system still met engineering standards that stress accuracy, reliability, fairness, maintainability, and the ability to be used in the real world.

5.1.1 Software Standards

The project used widely accepted software standards during the development and testing phases. Python was the main programming language because it is easy to read and has a strong ecosystem for machine learning research. The models were built, trained, and tested using frameworks like TensorFlow and Keras, which offered GPU acceleration, high-level APIs, and a lot of community support. These frameworks made sure that the models could be used in the same way and reproduced in different settings. Libraries like NumPy and Pandas made it easy to do math, clean up data, and change arrays, which were all important for working with large image datasets.

We used visualization tools like Matplotlib to make graphs for accuracy, loss, confusion matrices, and Grad-CAM heatmaps. For bigger models like DenseNet121 and Xception, CUDA and cuDNN made sure that training was faster on the GPU. Git and GitHub helped keep track of changes during different stages of experimentation and made it easier to keep clear records of model updates, dataset changes, and training procedures. These software standards made sure that the project stayed stable, manageable, and scalable, and they made it easy for people to work together and make improvements in the future.

5.1.2 Hardware Standards

The hardware environment met the performance standards needed to train and test deep learning models. A setup with a GPU was necessary to meet the computing needs of CNN architectures, especially DenseNet121 and ResNet50, which have millions of parameters. The T4 GPU in Google Colab sped up training, cut down on computation time, and made it possible to process larger batches. Having enough RAM made sure that data loading, augmentation pipelines, and model training all ran smoothly without any slowdowns.

Storage standards were also taken into account because the dataset, model checkpoints, logs, and output visualizations all needed a lot of memory. It was important to have fast and reliable storage so that checkpoints could be saved correctly and training could be picked up where it left off if needed. This hardware setup was in line with widely accepted standards for machine learning experiments, which meant that model training, testing, and comparison were all consistent, fast, and reliable.

5.1.3 Communication Standards

It was important to have communication and collaboration standards in place to manage the project's workflow, keep track of experiments, and get the system ready for possible integration into mobile or web platforms. GitHub was the main way for people to talk to each other and work together on code management. It made it easy to keep track of all changes, keep track of issues, and keep track of all changes. We used Google Colab notebooks to show experiments in a clear, interactive way. Each notebook had code, results, graphs, and explanations all in one

place, making it easy to share with coworkers or supervisors.

The system uses standard communication protocols like REST API compatibility, JSON data formatting, and secure HTTPS connections. This makes it easy to integrate into mobile apps or cloud-based platforms in the future. These communication standards make the system easier to maintain and add to by making it clear, traceable, and organized.

5.2 Impact on Society, Environment and Sustainability

The coconut disease detection system could greatly improve the health of the environment, the community, and agriculture. Farmers can take action quickly and stop the spread of diseases like Bud Rot, Yellow Leaf Disease, and Leaf Rot if they can identify them early and correctly. This could save entire plantations from damage. Finding problems early means you don't have to use as many pesticides, which is good for the environment and your health. Using less pesticides makes the soil cleaner, the water sources safer, and the crops healthier.

The system is useful for small-scale farmers who don't always have access to agricultural experts or lab analysis. The system gives farmers in rural areas the power to make smart choices by letting them quickly and accurately identify diseases right from their smartphones. This helps keep incomes stable, lowers crop losses, and makes food more secure in the area. From a sustainability point of view, the system encourages precision farming, which means that treatments and interventions are only used when they are needed. This cuts down on waste, pollution, and helps keep the environment in balance over the long term.

5.2.1 Impact on Life

The system has important effects on making life better for people, especially in rural farming areas. Farmers can find diseases early enough to stop big production losses by giving them a cheap and easy-to-use diagnostic tool. This not only protects their income, but it also helps the agricultural economy stay stable in areas where coconut farming is a major source of income. Because it works well on lightweight models like MobileNetV2, it can run on basic smartphones. This means that even farmers who can't afford more advanced devices can use it.

Cutting down on the use of unnecessary pesticides is good for people's health as well. Farmers, families, and people who live nearby are less likely to come into contact with harmful chemicals, which lowers the risk of long-term health problems. Healthier crops make food safer to eat and lower the risk of contamination. The system makes life better by making the environment safer, the economy more stable, and health better, which makes farming a more stable and supportive way of life.

5.2.2 Impact on Society & Environment

The system helps sustainable agricultural development on a larger scale by bringing modern

AI-based tools into traditional farming methods. It makes disease management more accurate and based on data, which cuts down on the trial-and-error method that often causes unnecessary crop damage or too much chemical use. This change can make food more available, help local economies, and make it easier for agricultural support services to do their jobs. The effect on the environment is just as big. By using pesticides less, we can protect soil fertility, keep biodiversity, and keep groundwater and nearby ecosystems from getting polluted. Bees, earthworms, and other beneficial insects are some of the organisms that are important for maintaining ecological balance. They do better when they are not exposed to as many chemicals. So, the system helps both farming and protecting the environment.

5.2.3 Ethical Aspects

When designing and putting the system into use, it was important to think about ethics. The dataset used for training came from publicly available repositories, so it didn't contain any private or sensitive information. By balancing the dataset and using augmentation and oversampling techniques to reduce bias, fairness and neutrality were kept. Using Grad-CAM visualizations helped farmers understand why the model predicted a certain disease, which made deep learning less of a "black box" issue. We chose lightweight architectures for deployment to cut down on energy use, which helped reduce the environmental impact of AI models. The system was made to be easy to use and cheap so that small farmers aren't left behind because they can't afford it. These moral concerns make sure that the system stays responsible, fair, and helpful for the communities it wants to help.

5.2.4 Sustainability Plan

The plan for sustainability is to build a disease detection system that is long-lasting, flexible, and cheap. The system can run well on smartphones without needing high-end hardware because it uses lightweight architectures like MobileNetV2. This makes it perfect for rural and remote areas. With cloud-based configurations, users don't have to reinstall or retrain anything when new disease images become available. This keeps the models accurate over time. The system can be added to mobile apps or agricultural advisory services to make money through subscriptions, partnerships, or institutional licensing. This will help the economy stay strong. Training programs, support in multiple languages, and easy-to-use interfaces help farmers use the technology easily, which helps social sustainability. To protect the environment, we need to use fewer chemicals and support more precise farming methods. This plan makes sure that the system stays useful, relevant, and important for many years.

5.3 Project Management and Financial Analysis

The project was organized into steps, such as gathering data, cleaning it up, training the model, testing it, and getting it ready to be used. The financial analysis looks at both a low-budget and a high-budget approach. A minimal budget uses free tools like Google Colab and already-owned hardware, which is fine for school or small projects. The

extended budget includes dedicated GPUs, paid cloud resources, mobile app development, and long-term storage. This makes it perfect for use by businesses or the government. Subscriptions to mobile apps, service-based pricing for farmers, institutional licensing for agricultural departments, or partnerships with agricultural research centers are all examples of practical revenue models. These financial options help make sure that the project stays possible and can grow over time, while also giving farmers access to affordable and reliable technology.

5.3.1 Low-Cost Research Setup

In this project we have used free platforms and open-source tools like Google Drive, Github, and Colab Notebook for training and storage. This is the best way to do academic research. However, there were also some small costs that had to be paid for this project.

Table 5.1: Minimum viable budget estimation

Item	Cost (USD)
GPU-(Colab T4)	50 \$
Data Storage & Backup	10 \$
Software	10 \$
Internet & Utilities	30 \$
Others	30 \$
Total Estimated Budget	130 \$

5.3.2 Advanced Computational Setup

Scaling the system for high volume training, commercial use, or long-term research will cost a lot of money. An advanced computational setup will need high-performance hardware, software, and other costs.

Table 5.2: Advanced computational setup budget estimation

Item	Cost (USD)
High Performance GPU	1000 - 1200 \$
High-performance workstation	2000 - 2500 \$

Storage & Backup	1500 - 2000 \$
Internet & Utilities	100 - 200 \$
Others	200 - 500 \$
Total Estimated Budget	4000 - 6000 \$

5.3.3 Potential Funding Sources

There are many practical and effective ways to fund the proposed coconut disease detection system. There are a few main ways to get money for the project, and each one can help make it possible, scalable, and available to farming communities.

5.3.3.1 Government Funding

Digital agriculture, smart farming, and rural development programs run by the government can be a big source of money. These projects often support technologies that boost productivity, make food safer, and have less of an effect on the environment. This makes this project a good candidate for agricultural innovation grants and subsidies.

5.3.3.2 Private Sector Sponsorship

Working with companies that make farm equipment, agricultural technology, or agribusiness can help with money and technology. These businesses often buy tools that help them keep an eye on their crops and deal with diseases. This is because these systems fit with their long-term business goals and plans for innovation.

5.3.3.3 Crowdfunding

Crowdfunding is a way to get money from people in the community, especially those who care about sustainable farming and new technologies. People's interest in AI solutions for farming can help raise enough money to pay for the development phase.

5.3.3.4 Non-profit Organizations

NGOs that work on food security, rural development, and reducing poverty may pay for this project because it will directly help farmers make more money and improve their crop yields. They usually back low-cost, high-impact tech solutions that can be used in communities that don't have enough resources.

5.3.4 Return on Investment (ROI)

You can look at the return on investment for this system in terms of both short-term and

long-term benefits to society and the environment. Farmers can clearly see how the system will help them make more money. The project not only makes money, but it also has a lot of other benefits. It helps farming become more sustainable, encourages the use of new technologies in farming, and helps ensure food security in the long term. The return on investment (ROI) is also affected by how many people use the system, how easily it can be adapted to different areas, and how well it can work with local farming processes. When used in many communities, the total effect on productivity, the health of the environment, and the well-being of farmers becomes very large. The advantages of using this system are likely to be greater than the costs of creating and implementing it. This is because it will lead to long-term increases in crop yields, less use of chemicals, and greater overall sustainability. The system is a good long-term investment for both partners and beneficiaries if it gets the right amount of money, the right resources, and grows slowly.

5.3.4.1 Better Pest Management

The system makes it possible to find coconut tree diseases early, which greatly lowers crop loss and raises yearly yield. For instance, a farmer who manages crops worth \$5,000 could make an extra \$500 to \$800 each season by stopping the spread of disease. Using less pesticides also lowers the cost of farming and makes the food better.

5.3.4.2 Technological, Environmental, and Social Gains

The system helps the environment in the long run by using fewer chemicals, making the soil healthier, and making food safer to grow. It promotes the use of new farming technologies, helps protect biodiversity, and makes food security better overall. Long-term ROI depends on a lot of people using it, being able to scale it up in different areas, and making farming more productive all the time.

5.4 Complex Engineering Problem

5.4.1 Complex Problem Solving

This project addressed complex engineering problems that needed people with different levels of technical knowledge, analytical skills, and practical design skills to work together. It is very hard to tell what kind of diseases affect coconut leaves. This is because the images can change, there are too many classes, and the hardware isn't very good. This is a complex problem to solve because it requires a lot of knowledge about deep learning architectures, careful preprocessing strategies, comparing different models, and thinking about how to deploy them in real time. The solution also involves picking the right models, finding the right balance between accuracy and cost, working with limited data, and making sure that Grad-CAM makes sense. These parts show that the project is a more

difficult engineering problem that requires both technical knowledge and a way to make decisions that is organized.

Table 5.3: Mapping with Complex Engineering Problem.

EP1 Dept of Knowledg e	EP2 Range Of Conflicting Requirement s	EP3 Depth of Analysi s	EP4 Familiarit y of Issues	EP5 Extent of Applicabl e Codes	EP6 Extent Of Stake- holder Involveme nt	EP7 Interdependen ce
✓	✓	✓	✓			✓

Justifications:

- **EP1 – Depth of Knowledge**

The project required strong expertise in deep learning, computer vision, and CNN-based architectures. Implementing transfer learning, data augmentation, class balancing, and model fine-tuning demanded detailed understanding of TensorFlow/Keras, GPU-based training, and evaluation metrics such as accuracy, loss, precision, recall, and F1-score. Additional knowledge of Grad-CAM was essential to produce visual explanations, making the system interpretable and practically deployable for coconut disease diagnosis. [ch-3][3.2]

- **EP2 – Range of Conflicting Requirements**

The system had to be very accurate and light enough to work in real time, especially on mobile devices and other hardware with limited resources. When we used both heavy models (DenseNet121, ResNet50) and light models (MobileNetV2, MobileNetV3), we had to make choices between performance and cost. The model also had to deal with an uneven dataset, make sure that data use was ethical, and work well in a variety of field conditions, like changes in lighting and noise in leaf images. [ch-4] [4.1, 4.2]

- **EP3 – Depth of Analysis**

An in-depth examination was conducted on all five models, encompassing accuracy/loss curves, confusion matrices, and precision, recall, and F1-scores for each class. A comparative evaluation showed that there are trade-offs between speed and accuracy in computing. We chose MobileNetV2 as the best-performing model because it has a high accuracy rate (98.7%) and a lightweight architecture, as shown by thorough quantitative and visual tests. [ch-4] [4.2.4]

- **EP4 – Familiarity of Issues**

The project dealt with common problems in agricultural image datasets, like poor image

quality, noise, background clutter, class imbalance, and mislabeled samples. To fix these problems, we used a number of preprocessing methods, such as cleaning, resizing, normalization, augmentation, and class balancing. The system was improved so that it could work in real-life situations where coconut leaves might look damaged, only partially visible, or be caught in bad lighting. [ch-3] [3.2.2]

- **EP7 – Interdependence**

There were many parts to the project that depended on each other, such as data preprocessing, model training, transfer learning, augmentation pipelines, evaluation frameworks, and explainability (Grad-CAM). Every module depended on the output of another. For instance, balanced datasets made models more stable, and well-trained CNNs made Grad-CAM visualization more reliable. This interconnected workflow made sure that the disease detection system worked well and was all in one place. [ch-3] [3.2]

Mapping with Knowledge Profile

Table 5.4: Mapping with knowledge Profile.

K1 Natural Science	K2 Mathematics	K3 Engineering Fundamentals	K4 Specialist Knowledge	K5 Engineering Design	K6 Engineering Practice	K7 Comprehension	K8 Research Literature
		✓	✓	✓	✓	✓	✓

Justifications:

- **K3 – Engineering Fundamentals**

The project used basic engineering ideas about machine learning, optimization, and evaluation. The basis for training and testing models were ideas like convolution, activation functions, overfitting control, loss minimization, and statistical accuracy measures. We also used basic engineering principles to compare CNN-based architectures (DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception) and figure out what they were good and bad at. [Ch. 3, Ch. 4]

- **K4 – Specialist Knowledge**

To build the system, advanced expertise is required at deep learning, especially CNNs, transfer learning, and AI that can be explained. To make a good coconut disease detection model, we need to understand model architectures, feature extraction, class balancing, data augmentation, and Grad-CAM. This level of knowledge goes beyond basic ML and shows expertise in using computer vision for farming. [Ch3] [3.2, 3.2.9]

- **K5 – Engineering Design**

The design of the entire workflow—from preprocessing data, choosing an augmentation strategy, selecting a model, and setting up training to evaluating it, needed to take into account both functional and non-functional factors. Optimized for accuracy, less work for the computer, and made sure the model could work in real-world situations, like on mobile devices or at the edge. The design choices took into account speed, resource use, how easy it is to understand, and how reliable the classification is. [Ch-3] [3.1, 3.2]

- **K6 – Engineering Practice**

The project followed strong engineering practices like cleaning datasets, making sure that training could be repeated, comparing models in a systematic way, keeping track of versions, and writing reports in a structured way. Model experimentation, fine-tuning, metric-driven evaluation, and Grad-CAM visualization are all parts of modern engineering workflows that are used in real-world AI deployments. The work also showed how to do things like keep records and try things out over and over again. [ch-4] [4.1, 4.2, 4.2.4, 4.2.5]

- **K8 – Research Literature**

A thorough review of previous studies on plant disease detection, CNN architectures, transfer learning, and explainable AI helped shape the methodology. The literature review helped find gaps in previous work, such as class imbalance, a lack of lightweight models, and a lack of interpretability. [Ch-2] [2.2, 2.3]

5.4.2 Engineering Activities

The project involved several key engineering activities, including dataset preparation, model training, systematic evaluation, and the design of an explainable deep learning pipeline for coconut disease detection. It was important to choose the right resources (EA1), work together and be supervised while developing (EA2), and carefully combine several CNN models with Grad-CAM explainability into a useful workflow. The project also looked at how it would affect the environment and society, like cutting down on pesticide use and promoting sustainable farming (EA4). Being familiar with machine learning tools and having taken similar classes before made it possible to test things out even with limited hardware (EA5). In general, the work shows a good mix of technical skill, responsible design, and knowledge of the real world.

Mapping with Complex Engineering Activities

Table 5.5: Mapping with Complex Engineering Activities.

EA1 Range of re- sources	EA2 Level of Interaction	EA3 Innovation	EA4 Consequences for society and environment	EA5 Familiarity
✓	✓		✓	✓

Justifications:

- **EA1 – Range of Resources**

The project used a wide range of tools and resources, including TensorFlow/Keras, Google Colab for GPU-based training, GitHub for version control, and visualization tools like Grad-CAM and Matplotlib. The project also used libraries for image processing, modules for augmentation, and more than one CNN architecture. This wide range of digital and computational tools shows that there is a lot of engineering going on. [ch-4] [4.1, 4.1.1, 4.1.2]

- **EA2 – Level of Interaction**

Conversations with peers, academic supervisors, and knowledge of how to find diseases in crops all had an effect on the design of the system. This shows interaction with multiple perspectives, which is a key requirement for EA2.

- **EA4 – Consequences for Society and Environment**

By cutting down on unnecessary pesticide use, protecting crop yield, and helping to ensure food security, the proposed detection system supports sustainable agriculture. It cuts down on pollution and makes the soil and water better by making people less reliant on chemicals. Faster disease detection saves farmers time, reduces crop loss, and improves their overall quality of life. These are all good things for people and the environment. [ch-5] [5.2]

- **EA5 – Familiarity**

We had to work with realistic limits, like a small dataset size, Colab's free tier's computational limits, different image quality, and the need to balance heavy and light architectures. Because you were already familiar with machine learning workflows, had taken classes on them, and had worked with CNNs and transfer learning before, you were able to quickly create prototypes and try out new ideas. This kept the project moving forward even though resources were limited. [Ch-3] [3.3], [ch-4] [4.1]

5.5 Summary

This chapter discussed the engineering standards, design choices, and effects on the real world of the coconut disease detection system that uses deep learning. It talked about how the project followed established standards for software, hardware, and communication to make sure the system stays reliable, efficient, and scalable in different deployment environments. The selected models and training methodologies adhered to established machine learning principles, encompassing structured preprocessing, performance assessment, and optimization designed to enhance accuracy while managing computational expenses. The chapter also talked about how the system could help society in important ways, especially for smallholder farmers who often don't have access to advanced diagnostic tools. The system helps make food safer, the economy more stable, and agriculture more resilient by cutting down on the use of pesticides that aren't needed, finding diseases earlier, and helping crops grow healthier. To reduce harm and build trust, ethical issues like fairness, openness, data privacy, and responsible model behavior were talked about. Lightweight model design, low-cost mobile and edge deployment, and ongoing community-driven feedback loops that keep the system relevant all contributed to sustainability. The chapter also talked about some of the biggest engineering problems that had to be solved, such as finding a balance between accuracy and limited resources, meeting real-time performance needs, and working around the limitations of datasets. These talks show how the project combines technical rigor with social responsibility and a long-term awareness of the environment.

Chapter 6

Conclusion

This chapter provides a summary of the research results, talks about how important the coconut tree disease detection system is, and describes its flaws and what it could do in the future. It gives an overview of the whole process of designing, training, and testing several deep learning models. It also talks about how the system could help agriculture, especially in areas where resources are limited.

6.1 Summary

This study presented a comprehensive deep learning framework aimed at classifying five categories of coconut tree diseases utilizing a newly assembled dataset from online sources. To improve quality and reduce bias before training, the dataset was carefully cleaned, resized, normalized, and augmented. We trained and tested five well-known convolutional neural networks DenseNet121, ResNet50, MobileNetV2, MobileNetV3, and Xception to find the best one for use in the real world. All of the models did very well, and DenseNet121 and ResNet50 had perfect precision, recall, and F1-scores. MobileNetV2, on the other hand, had the most balanced performance, with 99.8% training accuracy and 98.7% validation accuracy, and it also didn't need a lot of processing power. This made MobileNetV2 the best model for apps that run on mobile devices or at the edge. Grad-CAM was used on MobileNetV2 to make it easier to understand. It created heatmap visualizations that show the areas that affect the decision-making process. This step makes sure that the classification results are clear and trustworthy, especially for people who aren't tech-savvy and work in agriculture. The system as a whole shows a lot of promise for helping farmers, especially those in rural areas with few resources, by making it easier and faster to find diseases caused by pests. Farmers can use fewer pesticides, protect the quality of their coconut yield, and use more environmentally friendly farming methods if they find pests early on. Also taken into account were ethical issues like fairness, data safety, and accessibility. This made sure that the system is responsible, trustworthy, and good for long-term agricultural growth.

6.2 Limitation

Even though the research looks good on paper, there are still some issues that could create problems in the real world implementation. The dataset doesn't have a lot of natural variation in things like lighting, angle, and field conditions. This means that the model might have trouble when it sees real-world images that are very different from the training data. Even though augmentation helped balance the classes, synthetic transformations can't completely replace real field diversity. Also, DenseNet121 and ResNet50 are too heavy for low-power devices because they need a lot of memory and processing power. MobileNetV2 works well most of the time, but

it does have some problems with images that are noisy, blurry, or partially blocked. Finally the system may be suitable with light weight mobile devices but the system also needs clear, close-up photos from the user, which might be hard for farmers who use older phones.

6.3 Future Work

Future development can fix these problems and make the system work better and be easier to use. Adding more real farm pictures from different places, times of year, and weather conditions to the dataset would make the model much more flexible. Using advanced synthetic generation methods like GAN-based augmentation could make new variations and fix class imbalance more effectively. Techniques like quantization, pruning, and lightweight distillation could make MobileNetV2 even smaller and better for smartphones, edge devices, and offline settings without losing accuracy. Farmers would be able to quickly identify pests by uploading a picture or taking a picture with their phone if the model were built into a full-featured mobile app. If the system could work with IoT devices and automated monitoring tools, it could allow for ongoing real-time detection and warning systems for large plantations. Improvements to the user interface, like language options for different areas, voice-guided instructions, and easier ways to understand results, would make it much easier for rural farmers to use and adopt. Finally, the model will stay accurate, reliable, and relevant over time if we keep adding new field data and feedback to it. This is because disease patterns change over time.

